

CORE 01 — Digital Logic & Number Systems

Shared-core file 1 of 3. Study this once, get marks in all three electives.

Why this matters

Elective	Where it appears	Weight
Computer Forensic	TP-I §6 — "Logic, Formula, Unit etc." (Digital computers, logic gates, Boolean algebra, map simplification, combinational circuits, flip-flops, sequential circuits, decoders, multiplexers, registers, counters, memory unit, octal/hex/decimal/binary, 1's & 2's complement, floating point)	An entire numbered syllabus section — expect 1 Section-C (15 mark) question plus several MCQs
CS Degree	Paper-I §3 — "Logic Design" (Number system, binary arithmetic, Boolean algebra and logic functions, minimization, flip-flops, design of combinational and sequential circuits, registers, counters, decoders, encoders, adder circuits, code converters)	1 of only 6 Paper-I subjects → ~16% of Paper-I by topic count
CS Diploma	Paper-I §1.3 — data representation, BCD, ASCII, all four number systems and conversions, binary add/subtract, 1's and 2's complement	Guaranteed MCQs; conversions are the single most common Diploma MCQ type

Strategic note. This is the *cheapest* topic in the entire prep system. It is almost entirely mechanical: conversions, K-maps, adders, flip-flop tables. There is very little to "understand deeply" and a lot to simply *drill*. Given the MCQ paper has **negative marking of 1/3**, mechanical topics where you can be 100% certain of the answer are worth disproportionately more than topics where you'd be guessing. **Drill this to certainty.**

Time budget suggestion: 6–8 sessions of 3 hours. Days 1–2 number systems + complement arithmetic (pure drill), day 3 Boolean algebra + K-maps, day 4 combinational, day 5 flip-flops, day 6 counters/registers, day 7 sequential design, day 8 full revision + past paper attempt.

Topic checklist

- ☐ 1. Positional number systems; binary, octal, decimal, hexadecimal
- ☐ 2. All 12 conversion directions (including fractional parts)
- ☐ 3. BCD (8421), packed/unpacked, BCD addition with +6 correction
- ☐ 4. Excess-3, Gray code, and code conversion
- ☐ 5. ASCII, EBCDIC, Unicode — sizes and key code points
- ☐ 6. Binary arithmetic: add, subtract, multiply, divide
- ☐ 7. Signed representations: sign-magnitude, 1's complement, 2's complement
- ☐ 8. Complement subtraction and overflow detection
- ☐ 9. IEEE 754 single and double precision (encode + decode both directions)
- ☐ 10. Boolean algebra: postulates, laws, duality
- ☐ 11. De Morgan's theorems (2-variable and n-variable)
- ☐ 12. Logic gates, truth tables, universal gates (NAND/NOR realisations)
- ☐ 13. Canonical forms: minterms, maxterms, SOP, POS, conversion between them
- ☐ 14. K-map minimisation: 2, 3, 4 variables; SOP and POS; don't-cares
- ☐ 15. Half adder, full adder, ripple-carry adder, carry look-ahead (concept)
- ☐ 16. Half subtractor, full subtractor

- [] 17. Decoders and encoders (incl. priority encoder)
- [] 18. Multiplexers and demultiplexers; function implementation using MUX
- [] 19. Code converters (binary ↔ Gray, BCD ↔ Excess-3, BCD-to-7-segment)
- [] 20. Comparators, parity generator/checker
- [] 21. Latches vs flip-flops; SR, JK, D, T
- [] 22. Characteristic tables, characteristic equations, excitation tables
- [] 23. Race-around condition and master–slave JK
- [] 24. Registers; shift registers (SISO/SIPO/PISO/PIPO), universal shift register
- [] 25. Ring counter and Johnson (twisted-ring) counter
- [] 26. Asynchronous (ripple) counters and mod-N design
- [] 27. Synchronous counter design using JK / T flip-flops
- [] 28. Mealy vs Moore machines; state diagrams, state tables, state reduction
- [] 29. Full synchronous sequential design procedure (worked end-to-end)
- [] 30. Logic families and IC basics (TTL/CMOS, SSI/MSI/LSI/VLSI, fan-in/fan-out)

PART A — NUMBER SYSTEMS AND CODES

A1. Positional number systems

Concept — plain English first

Every number system you will ever meet in this exam works the same way, and if you internalise *one* idea you never have to memorise a conversion rule again.

A number is written as a string of digits. **Each digit's position gives it a weight, and the weights are powers of the base.** In decimal (base 10) the weights going left from the decimal point are 1, 10, 100, 1000 — that is 10^0 , 10^1 , 10^2 , 10^3 . Going right they are 10^{-1} , 10^{-2} , 10^{-3} . That's all "positional notation" means. Change the base to 2 and the weights become 1, 2, 4, 8, 16...; change it to 16 and they become 1, 16, 256, 4096.

So the *value* of any number is just "each digit multiplied by its weight, all added up":

$$N = d_{(n-1)} \cdot b^{(n-1)} + \dots + d_1 \cdot b^1 + d_0 \cdot b^0 + d_{(-1)} \cdot b^{(-1)} + \dots$$

Once you see it this way, **converting anything to decimal is one formula**, and the only thing you have to learn separately is how to go the *other* way (decimal → base b), which is repeated division for the integer part and repeated multiplication for the fraction.

Formal definition

A **positional number system of radix (base) b** represents a number as an ordered string of digits from the digit set $\{0, 1, \dots, b-1\}$, where the digit in position i (counting from 0 at the radix point, increasing leftward) carries weight b^i .

Key points

Base	Name	Digits	Digits needed per 1 hex digit	Common use
2	Binary	0,1	—	Machine level, logic circuits
8	Octal	0–7	1 octal = 3 binary bits	UNIX file permissions
10	Decimal	0–9	—	Human

16	Hexadecimal	0–9, A–F	1 hex = 4 binary bits	Memory addresses, MAC, colour, forensic hex dumps
----	-------------	----------	-----------------------	---

- Hex letters: **A=10, B=11, C=12, D=13, E=14, F=15**.
- With n digits in base b you can represent b^n distinct values, range **0 to b^n-1** .
- 4 bits = 1 **nibble**; 8 bits = 1 **byte**; a **word** is machine dependent (16/32/64 bits).
- Largest n -bit unsigned value = $2^n - 1$. (8-bit → 255, 16-bit → 65535, 32-bit → 4294967295.)
- To store a decimal number of D digits you need approximately **$D \times 3.32$** bits (since $\log_2 10 \approx 3.3219$).

Powers of 2 — memorise this table cold

2^0	2^1	2^2	2^3	2^4	2^5	2^6	2^7	2^8	2^9	2^{10}
1	2	4	8	16	32	64	128	256	512	1024

2^{11}	2^{12}	2^{13}	2^{14}	2^{15}	2^{16}	2^{20}	2^{30}	2^{40}
2048	4096	8192	16384	32768	65536	1 M	1 G	1 T

Also: $2^{-1} = 0.5$, $2^{-2} = 0.25$, $2^{-3} = 0.125$, $2^{-4} = 0.0625$, $2^{-5} = 0.03125$.

A2. Conversions — the complete method set

There are only **four techniques**. Everything else is a combination of them.

Technique 1 — Any base → Decimal: multiply by positional weights and sum

Worked example 1. $(1011.101)_2 \rightarrow$ decimal

Integer part: $1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 0 + 2 + 1 = 11$
 Fraction part: $1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} = 0.5 + 0 + 0.125 = 0.625$
 Answer: $(1011.101)_2 = (11.625)_{10}$

Worked example 2. $(725)_8 \rightarrow$ decimal

$7 \times 8^2 + 2 \times 8^1 + 5 \times 8^0 = 7 \times 64 + 2 \times 8 + 5 = 448 + 16 + 5 = (469)_{10}$

Worked example 3. $(2AF.C)_{16} \rightarrow$ decimal

$2 \times 16^2 + 10 \times 16^1 + 15 \times 16^0 + 12 \times 16^{-1}$
 $= 512 + 160 + 15 + 0.75 = (687.75)_{10}$

Technique 2 — Decimal → Any base: integer part by repeated DIVISION, fraction by repeated MULTIPLICATION

The rule that catches people: **integer remainders are read BOTTOM-to-TOP; fraction carries are read TOP-to-BOTTOM**. Write it on your rough sheet before you start.

Worked example 4. $(469)_{10} \rightarrow$ hexadecimal

$469 \div 16 = 29$ remainder 5 ↑
 $29 \div 16 = 1$ remainder 13 = D | read upward
 $1 \div 16 = 0$ remainder 1 |
 Answer: $(1D5)_{16}$
 Check: $1 \times 256 + 13 \times 16 + 5 = 256 + 208 + 5 = 469$ ✓

Worked example 5. $(11.625)_{10} \rightarrow$ binary

INTEGER 11: $\begin{array}{r} 11 \div 2 = 5 \text{ r } 1 \quad \uparrow \\ 5 \div 2 = 2 \text{ r } 1 \quad \\ 2 \div 2 = 1 \text{ r } 0 \quad \\ 1 \div 2 = 0 \text{ r } 1 \quad \end{array}$ → 1011	FRACTION 0.625: $\begin{array}{r} 0.625 \times 2 = 1.25 \rightarrow \text{carry } 1 \quad \downarrow \\ 0.25 \times 2 = 0.50 \rightarrow \text{carry } 0 \quad \\ 0.50 \times 2 = 1.00 \rightarrow \text{carry } 1 \quad \end{array}$ → .101
--	--

Answer: (1011.101)₂ ✓ matches Worked example 1

Exam warning on fractions: many decimal fractions never terminate in binary (e.g. $0.1_{10} = 0.0001100110011\dots_2$ recurring). If it does not terminate, the question will say "correct to n binary places" — stop after n multiplications and round. **This non-terminating property is exactly why floating-point arithmetic has rounding error,** a favourite MCQ link.

Technique 3 — Binary ↔ Octal / Hex: grouping (no arithmetic needed)

Because $8 = 2^3$ and $16 = 2^4$, each octal digit is exactly 3 bits and each hex digit exactly 4 bits. **Group from the radix point outwards**, padding the outer ends with zeros.

Worked example 6. $(1101011.1011)_2 \rightarrow$ octal and hex

OCTAL — groups of 3 from the point:

001	101	011	.	101	100	(padded: left "00", right "0")
1	5	3	.	5	4	→ (153.54) ₈

HEX — groups of 4 from the point:

0110	1011	.	1011	
6	B	.	B	→ (6B.B) ₁₆

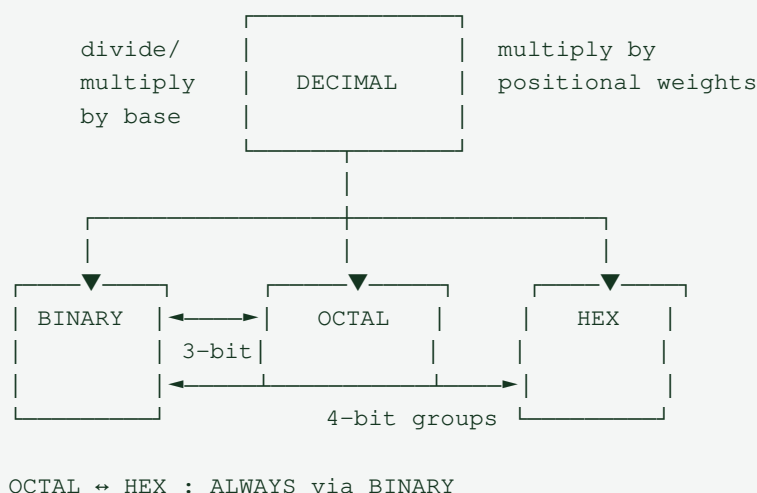
Worked example 7. $(3A7)_{16} \rightarrow$ binary → octal

$3 = 0011$, $A = 1010$, $7 = 0111 \rightarrow 0011\ 1010\ 0111$
 Regroup in 3s from the right: $001\ 110\ 100\ 111 \rightarrow (1647)_8$

Technique 4 — Octal ↔ Hex: always route through binary

Never try to convert octal to hex directly. 8 and 16 are not powers of each other in a convenient way. Go octal → binary → regroup → hex. (Worked example 7 shows the pattern in reverse.)

The full conversion map



Likely exam questions — Number systems

Q	Marks
Convert $(2AF.C)_{16}$ to binary, octal and decimal.	5
Convert $(0.6875)_{10}$ to binary and $(357.24)_8$ to hexadecimal, showing all steps.	5
Explain positional number systems. Convert $(1101.011)_2$ to decimal, $(89)_{10}$ to hex, and $(5C)_{16}$ to octal.	10
Discuss the four number systems used in digital computers, with conversion procedures between each pair, illustrating with examples. Why is hexadecimal preferred for representing memory addresses?	15

MCQ traps — Number systems

Trap	The right answer
"Read the remainders top-to-bottom"	Bottom-to-top for integer division; top-to-bottom for fraction multiplication. Examiners set the reversed string as a distractor.
Grouping binary from the <i>left</i>	Group from the radix point outwards . For integers that means from the right . $(1011)_2$ in octal is 001
Largest 8-bit number	Unsigned 255 (2^8-1), not 256. 256 is the <i>count</i> of values.
Number of distinct values in n bits	2^n values, range $0 \dots 2^n-1$.
$(11)_2$ vs $(11)_8$ vs $(11)_{16}$	3, 9, 17 respectively. Read the subscript.
"Octal to hex directly by grouping"	False. Must go via binary.
Is 1010 a valid BCD digit?	No — see A3.

A3. BCD (Binary Coded Decimal)

Concept

Pure binary is great for machines and horrible for humans reading a 7-segment display or a cash register. **BCD is a compromise: encode each decimal digit separately in 4 bits.**

So 47 is *not* 101111 (pure binary); it is **0100 0111** — a "4" nibble followed by a "7" nibble. The price you pay is wasted space (6 of the 16 nibble patterns are illegal) and awkward arithmetic. The benefit is trivially easy conversion to and from a decimal display.

Key points

- The standard BCD code is **8421 BCD** (the weights of the four bits).
- Codes **1010, 1011, 1100, 1101, 1110, 1111 (10–15) are INVALID / forbidden** in BCD. Only 0000–1001 are legal. **Six unused codes** — very common one-mark MCQ.
- Unpacked BCD**: one decimal digit per byte (upper nibble is 0 or a zone). **Packed BCD**: two decimal digits per byte. Packed is twice as space-efficient.
- BCD is **less efficient** than pure binary: 4 bits carry only 10 of a possible 16 values, so BCD storage is roughly **20% larger** than binary for the same range.
- BCD is **not** the same as binary. $(25)_{10} = (11001)_2$ but $\neq (0010\ 0101)_{\text{BCD}}$.

Worked example 8 — BCD addition with the +6 correction

Rule: add the BCD digits as ordinary 4-bit binary. **If a nibble result exceeds 1001 (9), or if a carry out of that nibble occurred, add 0110 (6) to that nibble** and propagate the carry to the next digit.

Why +6? Because the nibble is naturally a mod-16 counter but BCD needs mod-10. You have to skip over the six illegal codes, so you push the value forward by 6.

Add 47 + 38 in BCD:

```

    0100 0111      (47)
+   0011 1000      (38)
-----
    0111 1111      → low nibble 1111 = 15 > 9, INVALID → add 0110
      1111
      + 0110
      -----
    1 0101          → low digit = 0101 (5), carry 1 into high nibble
high: 0111 + 1 = 1000 (8) → valid, no correction needed
-----
  1000 0101  = 85  ✓

```

Add 9 + 8 in BCD (single digit, generates a carry):

```

1001 + 1000 = 1 0001 → carry out of nibble occurred → add 0110 to the sum nibble
0001 + 0110 = 0111
Result: 0001 0111 = 17  ✓

```

MCQ traps — BCD

- **How many 4-bit codes are unused in BCD?** → 6.
- **What is added for BCD correction?** → 6 (0110), not 10.
- $(128)_{10}$ in BCD requires **12 bits** (3 digits $\times$ 4), in pure binary only 8.
- BCD 0001 0000 = **10**, not 16.

A4. Excess-3, Gray code, and other codes

Excess-3

Concept: Take the BCD code and add 3 to every digit. Sounds pointless — but it makes the code **self-complementing**: the 9's complement of a decimal digit is obtained by simply inverting all four bits (1's complement). That property made subtraction trivial in early decimal computers.

Decimal	BCD (8421)	Excess-3	Gray
0	0000	0011	0000
1	0001	0100	0001
2	0010	0101	0011
3	0011	0110	0010
4	0100	0111	0110
5	0101	1000	0111
6	0110	1001	0101

7	0111	1010	0100
8	1000	1011	1100
9	1001	1100	1101

- Excess-3 is **unweighted** and **self-complementing**. So are 2421 and 84-2-1.
- Verify self-complementing: 4 → 0111; invert → 1000 = Excess-3 for 5; and 9-4 = 5. ✓

Gray code (reflected binary code)

Concept: In Gray code, **consecutive values differ in exactly one bit**. Why care? Think of a rotary shaft encoder. If the position goes from 0111 to 1000 in pure binary, all four bits change at once, and if the sensors are a hair out of alignment you can momentarily read a wildly wrong value like 1111. With Gray code only one bit ever changes, so the worst error is off-by-one. Gray code is also the ordering used along the axes of a **Karnaugh map** — that is precisely why adjacent K-map cells differ in one variable.

Binary → Gray: keep the MSB; each subsequent Gray bit = XOR of adjacent binary bits.

$$g_{n-1} = b_{n-1}, \quad g_i = b_{i+1} \oplus b_i$$

Gray → Binary: keep the MSB; each subsequent binary bit = previous *binary* bit XOR current Gray bit.

$$b_{n-1} = g_{n-1}, \quad b_i = b_{i+1} \oplus g_i$$

Worked example 9. Binary 1011 → Gray

```

b3 b2 b1 b0 = 1 0 1 1
g3 = b3           = 1
g2 = b3 ⊕ b2 = 1⊕0 = 1
g1 = b2 ⊕ b1 = 0⊕1 = 1
g0 = b1 ⊕ b0 = 1⊕1 = 0
Gray = 1110

```

Back: Gray 1110 → binary

```

b3 = g3           = 1
b2 = b3 ⊕ g2 = 1⊕1 = 0
b1 = b2 ⊕ g1 = 0⊕1 = 1
b0 = b1 ⊕ g0 = 1⊕0 = 1
Binary = 1011 ✓

```

Other codes worth one line each

Code	Property
2421 (Aiken)	Weighted, self-complementing
84-2-1	Weighted (with negative weights), self-complementing
Biquinary	7 bits, 2 ones per code, error detecting
Parity (even/odd)	Adds 1 bit; detects any odd number of bit errors; cannot correct
Hamming code	Adds k parity bits where $2^k \geq m+k+1$; detects 2, corrects 1 bit error
Alphanumeric	ASCII, EBCDIC, Unicode

A5. ASCII, EBCDIC, Unicode

Key points

Code	Bits	Characters	Notes
ASCII	7 (often stored in 8 with parity/extension)	128	American Standard Code for Information Interchange
Extended ASCII	8	256	Adds box-drawing, accented characters
EBCDIC	8	256	IBM mainframes
Unicode	16-bit (UCS-2) / variable (UTF-8: 1–4 bytes)	65,536+ / >1,000,000 code points	UTF-8 is backward compatible with ASCII

Code points to memorise (they get asked directly):

Character	Decimal	Hex	Binary
NUL	0	00	0000000
Space	32	20	0100000
0	48	30	0110000
9	57	39	0111001
A	65	41	1000001
Z	90	5A	1011010
a	97	61	1100001
z	122	7A	1111010
DEL	127	7F	1111111

- **Difference between an uppercase and its lowercase letter is exactly 32 (0x20)** — i.e. only bit 5 differs. That's how a case-toggle is a single OR/AND with 0x20.
- Digits: ASCII value = digit + 48. So converting a character digit to its numeric value is `c - '0'`.
- ASCII codes 0–31 and 127 are **control characters** (non-printing). 32–126 are printable.

Forensic relevance (for the CF paper): hex dumps in forensic tools display file contents as hex bytes plus an ASCII column. File signatures / magic numbers are read this way — e.g. `FF D8 FF` = JPEG, `25 50 44 46` = %PDF, `50 4B 03 04` = ZIP/DOCX/XLSX, `89 50 4E 47` = PNG. Being fluent in hex↔ASCII is a directly examinable forensic skill.

PART B — BINARY ARITHMETIC AND SIGNED NUMBERS

B1. Unsigned binary arithmetic

Addition and subtraction rules

ADDITION	SUBTRACTION
$0 + 0 = 0$	$0 - 0 = 0$
$0 + 1 = 1$	$1 - 0 = 1$
$1 + 0 = 1$	$1 - 1 = 0$
$1 + 1 = 0$ carry 1	$0 - 1 = 1$ borrow 1
$1 + 1 + 1 = 1$ carry 1	

Worked example 10. $1011 + 1101$

```
  1111  ← carries
  1011  (11)
+ 1101  (13)
-----
 11000  (24) ✓
```

Worked example 11. $1101 - 0111$ by direct borrow

```
  1101  (13)
- 0111  (7)
-----
  0110  (6) ✓
```

Multiplication and division

Worked example 12. 1101×101 ($13 \times 5 = 65$)

```
  1101
×  101
-----
  1101      ← ×1
 0000      ← ×0, shifted
 1101      ← ×1, shifted twice
-----
1000001    = 64 + 1 = 65 ✓
```

Binary multiplication is just **shift and add**: for each 1 bit in the multiplier, add a shifted copy of the multiplicand. This is exactly what a hardware multiplier does.

Worked example 13. $1011 \div 10$ ($11 \div 2 = 5$ remainder 1)

```
    101      ← quotient
10 ) 1011
   10
   --
    01
    00
    --
     11
     10
     --
      1      ← remainder
Quotient 101 = 5, remainder 1 ✓
```

B2. Signed number representations

Concept

Binary has no minus sign, so we have to *spend a bit* on the sign. There are three schemes, and understanding **why 2's complement won** is the point of the whole topic.

Imagine an 8-bit odometer. It counts 0, 1, 2 ... 254, 255, and then rolls over to 0. Now suppose we just *declare* that the top half of the dial (128–255) means the negative numbers $-128 \dots -1$. Then "subtracting 1" is the same as "adding 255", because going forward 255 clicks on a 256-click dial lands you exactly one click back. **That is 2's complement.** Its enormous advantage: the adder circuit does not need to know or care about signs. One adder handles both addition and subtraction, and there is only one representation of zero. Sign-magnitude and 1's complement both have a wasteful **+0 and -0**.

The three schemes for 8 bits

Scheme	How to negate	Range (8-bit)	Zeros	Hardware
Sign-magnitude	Flip the MSB only	$-127 \dots +127$	Two (0000 0000, 1000 0000)	Needs separate sign logic; comparator required
1's complement	Invert every bit	$-127 \dots +127$	Two (0000 0000, 1111 1111)	Needs end-around carry
2's complement	Invert every bit, then add 1	$-128 \dots +127$	One	Simplest; universally used

General n-bit ranges:

Scheme	Range
Unsigned	0 to $2^n - 1$
Sign-magnitude	$-(2^{n-1} - 1)$ to $+(2^{n-1} - 1)$
1's complement	$-(2^{n-1} - 1)$ to $+(2^{n-1} - 1)$
2's complement	-2^{n-1} to $+(2^{n-1} - 1)$ — asymmetric, one extra negative

Same number, three ways (8-bit, value -20):

	Bits
+20	0001 0100
-20 sign-magnitude	1 001 0100
-20 1's complement	1110 1011
-20 2's complement	1110 1100

The two shortcuts for taking a 2's complement

1. **Invert-and-add-1** (safe, always works).
2. **Copy-from-the-right trick** (faster in the exam): scanning from the LSB, **copy every bit up to and including the first 1, then invert everything to the left of it.**

```

0001 0100      ← +20
    ^ first 1 from right is bit 2
copy "100", invert "00010" → "11101"
= 1110 1100    ← -20    ✓ (same as invert-and-add-1)

```

Key identity: the 2's complement of the 2's complement gives you back the original.

Interpreting a 2's complement number

If the MSB is 0, read it as a normal unsigned binary number. If the MSB is 1, the number is negative — **take its 2's complement to find its magnitude**, then attach the minus sign.

Equivalently, use the **weighted formula** (very useful for MCQs):

Value = $-d_{n-1} \cdot 2^{n-1} + \sum_{i=0 \text{ to } n-2} d_i \cdot 2^i$ (MSB has NEGATIVE weight)

Example: 1110 1100 = $-128 + 64 + 32 + 8 + 4 = -20$ ✓

B3. Subtraction by complements — worked both directions

The procedure (2's complement)

1. Represent both operands in n -bit 2's complement.
2. Take the 2's complement of the **subtrahend** (the number being subtracted).
3. **Add.**
4. **If there is a carry out of the MSB, discard it.** The result is correct and positive.
5. **If there is no carry out, the result is negative;** take its 2's complement to read the magnitude.

Worked example 14 — 40 – 25 (positive result), 8-bit 2's complement

```

40 = 0010 1000
25 = 0001 1001 → 2's complement = 1110 0111

  0010 1000
+ 1110 0111
-----
1 0000 1111 ← carry out of MSB = 1 → DISCARD it
  0000 1111 = 15    ✓ (40 - 25 = 15)

```

Worked example 15 — 25 – 40 (negative result), 8-bit 2's complement

```

25 = 0001 1001
40 = 0010 1000 → 2's complement = 1101 1000

  0001 1001
+ 1101 1000
-----
  1111 0001 ← NO carry out → result is negative
Take 2's complement of 1111 0001 → 0000 1111 = 15
Answer: -15    ✓

```

Worked example 16 — the same two sums in 1's complement (note the difference!)

```

40 - 25:  25 = 0001 1001 → 1's complement = 1110 0110

```

```

    0010 1000
+   1110 0110
-----
1 0000 1110    ← carry out = 1 → END-AROUND CARRY: add it back
    0000 1110
+           1
-----
    0000 1111 = 15 ✓

```

```

25 - 40:  40 = 0010 1000 → 1's complement = 1101 0111
          0001 1001
+         1101 0111
-----
          1111 0000    ← no carry → negative; 1's complement = 0000 1111 = 15 → -15 ✓

```

The single most examinable distinction: in **1's complement** a carry out must be **added back into the LSB** (end-around carry). In **2's complement** the carry out is simply **discarded**. Examiners love this.

Overflow detection

Overflow ≠ carry out. Overflow means the true result cannot fit in n bits.

Rules (all equivalent — learn at least two):

1. Overflow occurs **only when the two operands have the SAME sign and the result has the OPPOSITE sign**. Adding numbers of opposite sign can never overflow.
2. Overflow = **carry into the MSB** $\oplus$ **carry out of the MSB**. (This is the actual hardware detection circuit — a single XOR gate.)

Worked example 17 — overflow in 8-bit signed arithmetic. $100 + 50$

```

    0110 0100  (+100)
+   0011 0010  (+50)
-----
    1001 0110  = -106 in 2's complement !!
Both operands positive, result negative → OVERFLOW.
True answer 150 exceeds the +127 limit of 8-bit 2's complement.
Carry into MSB = 1, carry out of MSB = 0 →  $1 \oplus 0 = 1$  → overflow ✓

```

Likely exam questions — Binary arithmetic and complements

Q	Marks
Find the 1's and 2's complement of $(01101010)_2$ and of $(10110000)_2$.	5
Perform $(35)_{10} - (48)_{10}$ using 8-bit 2's complement arithmetic.	5
Distinguish between sign-magnitude, 1's complement and 2's complement representation. State the range of each for n bits and explain why 2's complement is preferred.	10
Explain the overflow condition in signed binary addition. Give the detection rule and illustrate with two 8-bit examples, one overflowing and one not.	10
Explain signed number representation in computers. Perform $45 - 78$ and $78 - 45$ using both 1's and 2's complement in 8 bits, clearly showing the treatment of the end carry in each case.	15

MCQ traps — Complements

Trap	Truth
Range of 8-bit 2's complement is $-127 \dots +127$	-128 to $+127$ (asymmetric)
2's complement has two zeros	One zero. 1's complement and sign-magnitude have two.
The carry out is "the answer is wrong"	In 2's complement a carry out is normal — discard it . Wrongness is signalled by <i>overflow</i> , which is different.
1's complement: discard the carry	No — add it back (end-around carry).
2's complement of 1000 0000 (8-bit)	It is itself ($1000\ 0000 = -128$); $-(-128)$ is unrepresentable. Classic trick question.
2's complement of 0000 0000	0000 0000, with a discarded carry.
"Carry and overflow are the same"	Different. Unsigned overflow = carry out; signed overflow = the sign rule.

B4. Floating point representation — IEEE 754

Concept

Fixed-point binary can't hold both 6.02×10^{23} and 1.6×10^{-19} in a sensible number of bits.

The fix is the same trick as scientific notation: **store the significant digits and the exponent separately**. In decimal we write 13.625 as 1.3625×10^1 . In binary we write $13.625_2 = 1101.101$ as **1.101101×2^3** .

Two clever design choices make IEEE 754 what it is:

1. **The hidden (implicit) bit**. In normalised binary scientific notation the leading digit is *always* 1 (there is nothing else it could be). So don't store it — you get one extra bit of precision for free. A 23-bit stored fraction gives **24 bits of actual precision**.
2. **Biased exponent**. Instead of storing the exponent in 2's complement, add a **bias** (127 for single, 1023 for double) so the stored field is always non-negative. Why? Because it makes floats **comparable as if they were plain integers** — the same magnitude-comparison hardware works, which is a big win.

Format table

	Single precision (float)	Double precision (double)
Total bits	32	64
Sign	1	1
Exponent	8	11
Mantissa/fraction stored	23	52
Effective precision (with hidden bit)	24 bits $\approx$ 7 decimal digits	53 bits $\approx$ 15–16 decimal digits
Bias	127	1023
Exponent range (unbiased)	$-126 \dots +127$	$-1022 \dots +1023$
Approx magnitude range	$\pm 1.18 \times 10^{-38} \dots \pm 3.4 \times 10^{38}$	$\pm 2.2 \times 10^{-308} \dots \pm 1.8 \times 10^{308}$

SINGLE PRECISION LAYOUT (hand-draw this)



Value = $(-1)^S \times 1.F \times 2^{(E - 127)}$ [for normalised numbers]

Special values — memorise this table, it is pure MCQ fodder

Exponent field	Fraction field	Meaning
All 0s (00000000)	All 0s	± Zero
All 0s	Non-zero	Denormalised (subnormal) number; value = $(-1)^S \times 0.F \times 2^{-126}$ (no hidden 1)
1 ... 254 (anything else)	any	Normalised number, value as above
All 1s (11111111)	All 0s	± Infinity
All 1s	Non-zero	NaN (Not a Number)

Worked example 18 — DECIMAL → IEEE 754 single precision: -13.625

Step 1 Sign: negative → S = 1

Step 2 Magnitude to binary:

13 = 1101

0.625 = .101 (0.625×2=1.25→1; 0.25×2=0.5→0; 0.5×2=1.0→1)

13.625 = 1101.101

Step 3 Normalise (move the point so exactly one 1 is on its left):

1101.101 = 1.101101 × 2³ → actual exponent = 3

Step 4 Biased exponent: E = 3 + 127 = 130 = 1000 0010

Step 5 Fraction = the bits AFTER the leading 1, padded to 23 bits:

101101 → 101 1010 0000 0000 0000 0000

Step 6 Assemble:

S Exponent Fraction

1 10000010 101101000000000000000000

Grouped into bytes: 1100 0001 0101 1010 0000 0000 0000 0000

Hex: C1 5A 00 00

Worked example 19 — IEEE 754 → DECIMAL: 0x42E48000

0x42E48000 = 0100 0010 1110 0100 1000 0000 0000 0000

S = 0 → positive

E = 1000 0101 = 133 → actual exponent = 133 - 127 = 6

F = 110 0100 1000 0000 0000 0000 → 1.F = 1.1100100100...

Value = $1.11001001 \times 2^6 = 1110010.01$
 = $64 + 32 + 16 + 0 + 0 + 2 + 0 . 0 + 1 \times 2^{-2}$
 = 114.25

Answer: +114.25

(Check: $114.25 = 1110010.01_2 = 1.11001001 \times 2^6$ ✓)

Key points

- The **mantissa is stored in sign-magnitude form**, NOT 2's complement. Only the exponent is biased.
- Normalised single-precision numbers have the mantissa in the range **$1.0 \leq 1.F < 2.0$** .
- **Adding two floats** requires: (a) align exponents by right-shifting the smaller mantissa, (b) add mantissas, (c) re-normalise, (d) round. **Multiplication** is easier: multiply mantissas, **add** exponents, subtract one bias.
- Floating point addition is **not associative**: $(a+b)+c \neq a+(b+c)$ in general, because of rounding. Classic MCQ.
- **Precision vs range trade-off**: more exponent bits → wider range but coarser precision; more mantissa bits → finer precision but narrower range. Total bits are fixed.

Likely exam questions — Floating point

Q	Marks
Represent -0.75 in IEEE 754 single-precision format.	5
What are denormalised numbers and why does IEEE 754 include them?	5
Explain the IEEE 754 single-precision format. Convert 87.375 to it and show the result in hexadecimal.	10
Explain floating point representation. Describe the single and double precision IEEE 754 formats, the role of the bias and the hidden bit, and the encoding of zero, infinity and NaN. Convert $(-25.375)_{10}$ to single precision and decode 0xC1C80000.	15

MCQ traps — Floating point

Trap	Truth
Bias for single precision is 128	127 ($= 2^{8-1} - 1$). Double is 1023 .
Mantissa is 24 bits stored	23 stored , 24 effective (hidden bit).
Hidden bit is stored in the fraction field	It is not stored at all ; it is implied.
Zero is represented with a hidden 1	Zero is the special case exponent = 0, fraction = 0 — the hidden bit rule is suspended.
Exponent stored in 2's complement	Biased (excess-127) representation , not 2's complement.
Double precision exponent is 12 bits	11 bits ; fraction is 52 . $1+11+52 = 64$ ✓

PART C — BOOLEAN ALGEBRA AND LOGIC GATES

C1. Boolean algebra

Concept

Ordinary algebra deals with numbers that can take infinitely many values. **Boolean algebra deals with variables that can take exactly two values, 0 and 1**, and three operations: AND ($\cdot$), OR ($+$), NOT ($'$). It was invented by George Boole in 1854 as a formalisation of logic; **Claude Shannon** realised in 1938 that switching circuits obey exactly the same algebra, and that insight is the foundation of all digital design.

The whole practical point is: *a circuit is an expression, an expression is a circuit*. If you can simplify the expression algebraically you have literally made the circuit smaller and cheaper. That is why "minimisation" is a syllabus item in its own right.

Postulates and laws — the complete table

Every law below has a **dual**, obtained by swapping AND $\leftrightarrow$ OR and $0 \leftrightarrow 1$. This is the **Principle of Duality** and it halves what you have to memorise.

#	Law	AND form	OR form (dual)
1	Identity	$A \cdot 1 = A$	$A + 0 = A$
2	Null / Dominance	$A \cdot 0 = 0$	$A + 1 = 1$
3	Idempotent	$A \cdot A = A$	$A + A = A$
4	Complement	$A \cdot A' = 0$	$A + A' = 1$
5	Involution	$(A')' = A$	(same)
6	Commutative	$A \cdot B = B \cdot A$	$A + B = B + A$
7	Associative	$(AB)C = A(BC)$	$(A+B)+C = A+(B+C)$
8	Distributive	$A(B+C) = AB + AC$	$A + BC = (A+B)(A+C)$ ← no decimal analogue!
9	Absorption	$A(A+B) = A$	$A + AB = A$
10	Absorption (2nd form)	$A(A'+B) = AB$	$A + A'B = A + B$
11	De Morgan	$(AB)' = A' + B'$	$(A+B)' = A' \cdot B'$
12	Consensus	$AB + A'C + BC = AB + A'C$	$(A+B)(A'+C)(B+C) = (A+B)(A'+C)$

Law 8's OR-form and Law 10 are the two that students forget and that exams exploit.

$A + A'B = A + B$ — verify: if $A=1$ both sides are 1; if $A=0$, LHS = $0 + 1 \cdot B = B$, RHS = $0+B = B$ ✓

De Morgan's theorems — the most important result in the topic

Statement (2 variables):

$(A \cdot B)' = A' + B'$	"NOT of AND = OR of NOTs"
$(A + B)' = A' \cdot B'$	"NOT of OR = AND of NOTs"

Statement (n variables / generalised):

$$(A_1 \cdot A_2 \cdot \dots \cdot A_n)' = A_1' + A_2' + \dots + A_n'$$

$$(A_1 + A_2 + \dots + A_n)' = A_1' \cdot A_2' \cdot \dots \cdot A_n'$$

The plain-English rule you should actually use: "Break the bar, change the sign."

To complement an entire expression: complement every variable **and** swap every AND with an OR and every OR with an AND (and swap 0s and 1s).

Proof by truth table (write this out if asked to "prove"):

A	B	AB	(AB)'	A'	B'	A'+B'	A+B	(A+B)'	A'B'
0	0	0	1	1	1	1	0	1	1
0	1	0	1	1	0	1	1	0	0
1	0	0	1	0	1	1	1	0	0
1	1	1	0	0	0	0	1	0	0

Columns (AB)' and A'+B' are identical; columns (A+B)' and A'B' are identical. **Hence proved.**

Why it matters practically: De Morgan is what lets you convert *any* circuit into a NAND-only or NOR-only circuit — which is what real ICs are built from because NAND is the cheapest gate in CMOS.

Worked example 20 — algebraic simplification

Simplify **F = A'B'C + A'BC + AB'C + ABC' + ABC**

$$\begin{aligned}
 F &= A'B'C + A'BC + AB'C + ABC' + ABC \\
 &= A'C(B' + B) + AB'C + AB(C' + C) && \text{[factor 1st two; factor last two]} \\
 &= A'C(1) + AB'C + AB(1) && \text{[complement law]} \\
 &= A'C + AB'C + AB \\
 &= C(A' + AB') + AB && \text{[factor C]} \\
 &= C(A' + B') + AB && \text{[law 10: } A' + AB' = A' + B'\text{]} \\
 &= A'C + B'C + AB
 \end{aligned}$$

Check by counting minterms: original covers m1,m3,m5,m6,m7. A'C = m1,m3; B'C = m1,m5; AB = m6,m7. Union = {1,3,5,6,7} ✓

Worked example 21 — applying De Morgan

Find the complement of **F = A(B + C'D)**

$$\begin{aligned}
 F' &= [A(B + C'D)]' \\
 &= A' + (B + C'D)' && \text{[De Morgan on the AND]} \\
 &= A' + B' \cdot (C'D)' && \text{[De Morgan on the OR]} \\
 &= A' + B' \cdot (C + D') && \text{[De Morgan on the inner AND, and involution } C''=C\text{]} \\
 &= A' + B'C + B'D'
 \end{aligned}$$

Duality vs complement — a distinction examiners test

	Dual	Complement
Operation	Swap AND↔OR, 0↔1	Swap AND↔OR, 0↔1, AND complement every variable
Example on F = A + B'C	F ^d = A(B' + C)	F' = A'(B + C')
Relationship	F ^d (A,B,C) = F'(A',B',C')	—

NOT	1
A+B	2
A·B	3
XNOR	4
XOR	5

Note the pleasing symmetry: NAND is cheap for AND, NOR is cheap for OR; XOR costs 4 NANDs but 5 NORs, and XNOR is the reverse.

Why they are universal: because {AND, OR, NOT} is a functionally complete set, and NAND can produce all three (rows 1–3 above). Same argument for NOR.

Converting any SOP circuit to all-NAND: replace every AND gate by a NAND and every OR gate by a "bubbled-OR" — which by De Morgan *is* a NAND. So a two-level AND-OR circuit maps one-for-one to a two-level NAND-NAND circuit with the same gate count. **Similarly, any two-level OR-AND (POS) circuit maps directly to NOR-NOR.**

IC / logic family facts (syllabus mentions "Integrated Circuits")

Scale	Gates per chip	Examples
SSI (Small Scale Integration)	1–10	7400 quad NAND, 7404 hex inverter
MSI (Medium)	10–100	Decoders, MUX, counters, registers (74138, 74151, 7490)
LSI (Large)	100–10,000	Small memories, early processors
VLSI (Very Large)	>10,000 (now billions)	CPUs, GPUs, SoCs
ULSI	>1,000,000	Modern microprocessors

Family	Full name	Speed	Power	Noise immunity
TTL	Transistor–Transistor Logic (74xx)	Fast	High	Moderate
CMOS	Complementary MOS (40xx, 74HCxx)	Moderate–fast	Very low static	High
ECL	Emitter Coupled Logic	Fastest	Highest	Low

- **Fan-in** = number of inputs a gate can accept. **Fan-out** = number of gate inputs one output can drive (typical TTL fan-out = 10).
- **Propagation delay** t_{pd} = time from input change to output change. Total delay of a circuit = delay along the **longest (critical) path**.
- **Noise margin** = the voltage cushion between a valid output level and the input threshold. CMOS has the best.

Common IC numbers worth knowing

IC	Function
7400	Quad 2-input NAND

7402	Quad NOR
7404	Hex inverter
7408	Quad AND
7432	Quad OR
7486	Quad XOR
7447	BCD to 7-segment decoder/driver
7474	Dual D flip-flop
7476	Dual JK flip-flop
7483	4-bit binary full adder
7485	4-bit magnitude comparator
7490	Decade ($\div 10$) counter
7493	4-bit binary ($\div 16$) ripple counter
74138	3-to-8 decoder
74147	10-to-4 priority encoder
74151	8-to-1 multiplexer
74153	Dual 4-to-1 MUX
74154	4-to-16 decoder
74157	Quad 2-to-1 MUX
74194	4-bit universal shift register

C3. Canonical forms: minterms, maxterms, SOP, POS

Concept

Any Boolean function can be written in two standard ("canonical") ways, and both come straight off the truth table.

- Look at every row where the output is **1**. For each such row write an AND term that is true *only* on that row. OR them all together. That's **Sum of Products (SOP)** and each AND term is a **minterm**.
- Look at every row where the output is **0**. For each write an OR term that is false *only* on that row. AND them all together. That's **Product of Sums (POS)** and each OR term is a **maxterm**.

Both describe the same function. SOP is usually smaller when the function has few 1s; POS when it has few 0s.

The rules for writing terms (get the polarity right — this is where marks are lost)

	Minterm (m_i)	Maxterm (M_i)
Formed from rows where $F =$	1	0
Variable appearing as 0 in the row	write complemented (A')	write uncomplemented (A)
Variable appearing as 1 in the row	write uncomplemented (A)	write complemented (A')
Terms combined by	OR	AND
Term itself is an	AND (product)	OR (sum)

Relationship: $M_i = m_i'$ and $m_i = M_i'$.

Therefore: $F = \sum m(a,b,c) = \prod M(\text{all other indices})$.

Worked example 22 — full canonical treatment of a 3-variable function

A	B	C	dec	F
0	0	0	0	0
0	0	1	1	1
0	1	0	2	1
0	1	1	3	0
1	0	0	4	0
1	0	1	5	0
1	1	0	6	1
1	1	1	7	1

Canonical SOP: $F = \sum m(1, 2, 6, 7) = A'B'C + A'BC' + ABC' + ABC$

Canonical POS: $F = \prod M(0, 3, 4, 5)$

- $M_0 (A=0, B=0, C=0) \rightarrow (A + B + C)$

- $M_3 (0, 1, 1) \rightarrow (A + B' + C')$

- $M_4 (1, 0, 0) \rightarrow (A' + B + C)$

- $M_5 (1, 0, 1) \rightarrow (A' + B + C')$

$F = (A+B+C)(A+B'+C')(A'+B+C)(A'+B+C')$

Note $\{1, 2, 6, 7\} \cup \{0, 3, 4, 5\} = \{0 \dots 7\}$ and the two sets are disjoint ✓

Terminology check

- **Canonical form** = every term contains **every** variable (standard/expanded form).
- **Standard form** = terms may have fewer variables (this is what minimisation produces).
So $AB + C$ is standard SOP but not canonical SOP.
- **Literal** = a variable or its complement. $A'BC$ has 3 literals.
- **Implicant** = any product term that implies F . **Prime implicant** = an implicant that cannot be combined with another to remove a literal. **Essential prime implicant** = a prime implicant that is the *only* one covering some particular minterm — **it MUST appear in the final answer.**

C4. Karnaugh map minimisation

Concept

Algebraic simplification works but you never know when you're finished, and it's easy to miss a step. **The K-map is a way of drawing the truth table so that simplification becomes visual pattern-matching instead of algebra.**

The trick is the ordering along the axes: **Gray code order (00, 01, 11, 10)**, not binary counting order. That guarantees **any two physically adjacent cells differ in exactly one variable** — and two minterms that differ in one variable can always be merged, cancelling that variable ($AB + AB' = A$). So "circle the adjacent 1s" *is* the algebra, performed geometrically.

The map also **wraps around** like a torus: the left column is adjacent to the right column, and the top row is adjacent to the bottom row. Forgetting this loses marks constantly.

The rules of grouping

1. Groups may contain only **1s** (for SOP) and must be **rectangular**.
2. Group sizes must be a **power of 2**: 1, 2, 4, 8, 16. Never 3, 5, 6.
3. **Make every group as LARGE as possible**. A group of 2^k cells eliminates k variables.
4. **Use as FEW groups as possible**.
5. Groups **may overlap** — that's fine and often necessary.
6. Groups **wrap around** the edges (and the four corners of a 4-var map form a valid quad).
7. Every 1 must be covered by at least one group.
8. A group that covers no *new* 1 is redundant — remove it.

Group size (4-var map)	Variables eliminated	Literals in term
1 (single cell)	0	4
2 (pair)	1	3
4 (quad)	2	2
8 (octet)	3	1
16 (all)	4	0 ($F = 1$)

Map layouts to hand-draw

3-variable map (A on rows, BC on columns):

		BC			
A	\	00	01	11	10
0		m0	m1	m3	m2
1		m4	m5	m7	m6

4-variable map (AB on rows, CD on columns) — **memorise this minterm layout**:

CD
AB \ 00 01 11 10

00	m0	m1	m3	m2
01	m4	m5	m7	m6
11	m12	m13	m15	m14
10	m8	m9	m11	m10

↑ adjacent ↑ (column wrap-around)

Row 00 and row 10 are also adjacent (row wrap-around)

Corners m0, m2, m8, m10 form a valid quad = $B'D'$

Worked example 23 — 4-variable SOP minimisation

Minimise $F(A,B,C,D) = \sum m(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$

CD
AB \ 00 01 11 10

00	1	1	0	1	m0, m1, m2
01	1	1	0	1	m4, m5, m6
11	1	1	0	1	m12, m13, m14
10	1	1	0	0	m8, m9

Group 1 — OCTET, the two left columns (CD = 00 and 01), all four rows:

cells m0,m1,m4,m5,m12,m13,m8,m9 = 8 cells. In all of them $C = 0$; A, B and D all vary.

→ term C'

Group 2 — QUAD, m0, m2, m4, m6 (rows AB=00 and 01, columns CD=00 and 10):

A = 0 throughout and D = 0 throughout. → term $A'D'$

Group 3 — QUAD, m4, m6, m12, m14 (rows AB=01 and 11, columns CD=00 and 10):

B = 1 throughout and D = 0 throughout. → term BD'

Check coverage: C' covers 0,1,4,5,8,9,12,13. $A'D'$ adds 2 and 6. BD' adds 14.

All 11 minterms covered ✓

Answer: $F = C' + A'D' + BD'$

Original canonical form had 11 terms $\times$ 4 literals = 44 literals. Minimised: 5 literals.

In the exam, say this — "reduced from 44 literals to 5" — it shows you understand the point.

Worked example 24 — K-map WITH DON'T-CARES

Minimise $F(A,B,C,D) = \sum m(1, 3, 7, 11, 15) + d(0, 2, 5)$

Concept of a don't-care: some input combinations can never occur (e.g. BCD inputs 1010–1111) or the output genuinely doesn't matter. Mark them **X**. You are then free to treat each X as either 0 or 1 — **whichever makes your groups bigger**. Never let an X force you to add an extra group; only ever use an X to *enlarge* a group you already need.

		CD				
AB \		00	01	11	10	
00		X	1	1	X	m0=X, m1=1, m3=1, m2=X
01		0	X	1	0	m5=X, m7=1
11		0	0	1	0	m15=1
10		0	0	1	0	m11=1

Group 1 — QUAD, column CD=11 (m3, m7, m15, m11): C=1 and D=1 throughout → **CD**

Group 2 — remaining uncovered 1 is m1. Group it with the don't-cares m0, m2, m3 to make the whole top row's left block a quad {m0, m1, m3, m2}: A=0 and B=0 throughout → **A'B'**

Answer: $F = CD + A'B'$

Alternative equally-valid answer: group m1 with m3, m5, m7 (using don't-care m5) giving {m1,m3,m5,m7} = **A'D**, so $F = CD + A'D$. Both are 4 literals and both are correct — say so in the exam if you spot it; minimal forms are not always unique.

What if we ignored the don't-cares? We'd get $F = CD + A'B'C'D$ — 6 literals instead of 4. That is exactly the marks-earning observation.

Worked example 25 — POS minimisation from a K-map

Minimise $F(A,B,C,D) = \sum m(0,1,2,5,8,9,10)$ in POS form.

Procedure: **group the ZEROS instead of the ones**, which gives you F' ; then apply De Morgan.

Zeros are at m3,4,6,7,11,12,13,14,15.

		CD			
AB \		00	01	11	10
00		1	1	0	1
01		0	1	0	0
11		0	0	0	0
10		1	1	0	1

Grouping the 0s:

- Column CD=11 (m3,7,15,11) → CD
- Row AB=11 (m12,13,15,14) → AB
- m4, m6, m12, m14 (rows 01,11 × cols 00,10) → BD'

$$F' = CD + AB + BD'$$

Apply De Morgan: **$F = (C' + D')(A' + B')(B' + D)$**

Sanity check with m5 (A=0,B=1,C=0,D=1): $(1+0)(1+0)(0+1) = 1 \cdot 1 \cdot 1 = 1$ ✓ (m5 is a 1)

Check m4 (0,1,0,0): $(1+1)(1+0)(0+0) = 1 \cdot 1 \cdot 0 = 0$ ✓ (m4 is a 0)

Prime implicants — the vocabulary questions

For $F = \sum m(0,1,2,4,5,6,8,9,12,13,14)$ from Worked example 23:

- **Prime implicants:** C', A'D', BD'
- **Essential prime implicants:** C' (only cover for m8, m1 etc.), A'D' (only cover for m2), BD' (only cover for m14) — here all three are essential.
- A **redundant prime implicant** is one whose minterms are all covered by others; it can be

dropped. A **selective (non-essential) prime implicant** is one where a choice exists.

Quine–McCluskey (tabulation) method — one paragraph is enough

K-maps become unusable beyond 5–6 variables because human pattern-recognition fails. The **Quine–McCluskey tabular method** does the same job algorithmically: group minterms by the number of 1s they contain, compare adjacent groups, combine any pair differing in one bit (marking both as used), repeat until no more combinations are possible; the unmarked terms are the prime implicants. Then build a **prime implicant chart** and select a minimal cover. It is systematic and **computer-programmable** — that is its advantage over K-maps. Its disadvantage is that it is tedious and the number of comparisons grows exponentially.

Likely exam questions — Boolean algebra and K-maps

Q	Marks
State and prove De Morgan's theorems using truth tables.	5
Simplify $F = A'B'C + A'BC + AB'C + ABC$ using Boolean laws.	5
What is a don't-care condition? Where do they arise?	5
Minimise $F(A,B,C,D) = \Sigma m(1,3,7,11,15) + d(0,2,5)$ using a K-map and draw the resulting circuit.	10
Explain the principle of duality. State all the postulates of Boolean algebra with their duals.	10
Define minterm, maxterm, implicant, prime implicant and essential prime implicant. Obtain both the minimal SOP and the minimal POS for $F(A,B,C,D) = \Sigma m(0,1,2,5,8,9,10)$.	15
Explain the Karnaugh map method of minimisation. Minimise $F(A,B,C,D) = \Sigma m(0,1,2,4,5,6,8,9,12,13,14)$, realise it using only NAND gates, and comment on the reduction achieved.	15

MCQ traps — Boolean algebra and K-maps

Trap	Truth
K-map columns are labelled 00,01,10,11	00, 01, 11, 10 — Gray code. This is the whole point of the map.
A group of 6 cells is allowed	Only powers of 2 : 1,2,4,8,16.
Corners of a 4-var map aren't adjacent	The four corners form a valid quad ($B'D'$). Wrap-around is real.
A quad eliminates 1 variable	A group of 2^k eliminates k variables. Quad (4 cells) eliminates 2 .
$A + BC = (A+B)(A+C)$ is false	It is true in Boolean algebra (distributive law's dual). No decimal analogue.
$A + A'B = A'B$	= A + B
Minterm m_5 for 3 vars is $A'BC'$	$m_5 = 101 = \mathbf{AB'C}$. Get the polarity rule right.
NAND count for XOR	4 NANDs. XNOR needs 5. (NOR: XNOR = 4, XOR = 5.)
Don't-cares must be treated as 1	Treat as whatever helps — 1 to enlarge a group, 0 otherwise. Never form a group <i>only</i> of Xs.
Every prime implicant must appear in the answer	Only the essential ones must.

$F = \Sigma m(0,2,5)$ means POS indices 0,2,5

POS uses **all the OTHER** indices.

PART D — COMBINATIONAL CIRCUITS

D0. Combinational vs sequential — the framing distinction

	Combinational	Sequential
Output depends on	Present inputs only	Present inputs and past history (state)
Memory	None	Has memory elements (flip-flops)
Clock	Not required	Usually clocked (synchronous)
Feedback	No feedback path	Feedback present
Described by	Truth table / Boolean expression	State table / state diagram / excitation table
Examples	Adder, MUX, decoder, encoder, comparator, code converter	Flip-flop, register, counter, shift register, RAM

Design procedure for combinational circuits (the 5 steps — quote these verbatim):

1. State the problem and determine the number of inputs and outputs.
2. Assign letter symbols to inputs and outputs.
3. Derive the **truth table**.
4. Obtain the **simplified Boolean expression** for each output (K-map / algebra).
5. Draw the **logic diagram** and verify it.

D1. Half adder and full adder

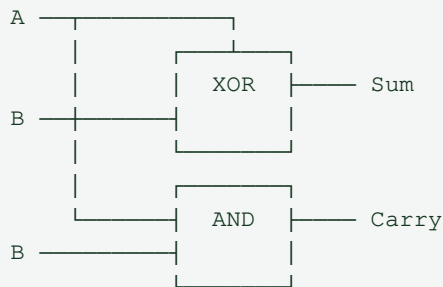
Half adder

Concept: Add two single bits. Two bits can sum to 0, 1 or 2 — and 2 doesn't fit in one bit, so you need a **Sum** output and a **Carry** output. "Half" because it cannot accept a carry *in* from a previous stage, so you can't chain them.

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$$\text{Sum} = A \oplus B = A'B + AB'$$

$$\text{Carry} = A \cdot B$$



Gate count: 1 XOR + 1 AND (or 5 NAND gates).

Full adder

Concept: To add multi-bit numbers you need each column to accept the carry from the column to its right. A full adder has **three inputs** (A, B, Cin) and two outputs (Sum, Cout). Three input bits can total 0–3, which is exactly what 2 output bits express.

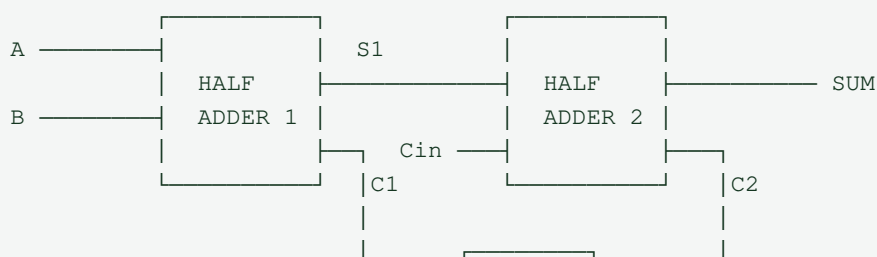
A	B	Cin	Sum	Cout	Notice
0	0	0	0	0	
0	0	1	1	0	
0	1	0	1	0	
0	1	1	0	1	
1	0	0	1	0	
1	0	1	0	1	
1	1	0	0	1	
1	1	1	1	1	Sum = 1 when an ODD number of inputs are 1

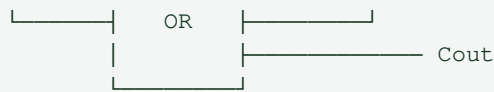
$$\text{Sum} = \Sigma m(1,2,4,7), \text{Cout} = \Sigma m(3,5,6,7)$$

K-map for Sum: the 1s form a **checkerboard** — no adjacent cells — so **Sum cannot be simplified below the XOR form**. That is a nice observation to state in an exam.

$$\begin{aligned} \text{Sum} &= A \oplus B \oplus \text{Cin} \\ \text{Cout} &= AB + B\text{Cin} + A\text{Cin} && \text{(majority function — 1 when 2 or more inputs are 1)} \\ &= AB + \text{Cin}(A \oplus B) && \text{(the form used when building from two half adders)} \end{aligned}$$

Implementation using two half adders + one OR gate — draw this:

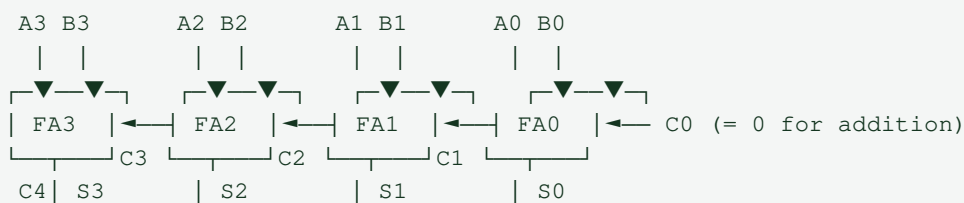




Gate count: 2 XOR + 2 AND + 1 OR (= 9 NAND gates).

4-bit ripple carry adder

Chain four full adders, carry-out of each feeding carry-in of the next.

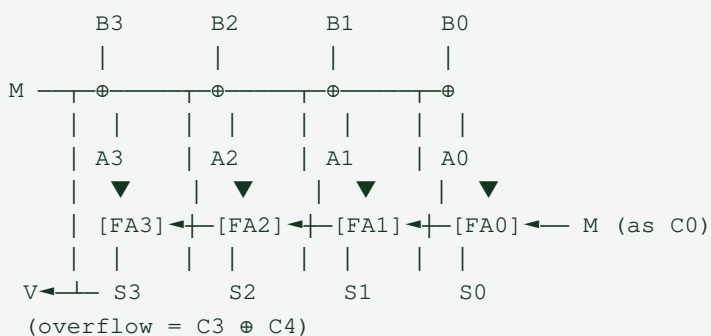


- **Problem: propagation delay.** The MSB sum is not valid until the carry has *rippled* all the way through. For n stages the worst-case delay is $\approx 2n$ gate delays (each FA contributes ~ 2 levels to the carry path). A 32-bit ripple adder is slow.
- **Solution: Carry Look-ahead Adder (CLA).** Define for each bit position:
 - **Generate** $G_i = A_i \cdot B_i$ (this stage definitely makes a carry)
 - **Propagate** $P_i = A_i \oplus B_i$ (this stage passes an incoming carry along)
 - Then $C_{i+1} = G_i + P_i \cdot C_i$, and expanding recursively gives **every** carry as a two-level function of the inputs. All carries appear simultaneously $\rightarrow$ **constant delay** (~ 3 gate delays) independent of width, at the cost of much more hardware and high fan-in.
- IC 74283 / 7483 are 4-bit CLA adders.

Adder/Subtractor combined circuit — a classic 10-mark question

Concept: Since $A - B = A + (2\text{'s complement of } B) = A + B' + 1$, you can build one circuit that does both. Feed each B_i through an **XOR with a control line M**, and feed M into C0 as well.

$M = 0 \rightarrow B_i \oplus 0 = B_i, C_0 = 0 \rightarrow$ circuit computes $A + B$
 $M = 1 \rightarrow B_i \oplus 1 = B_i', C_0 = 1 \rightarrow$ circuit computes $A + B' + 1 = A - B$



Overflow output $V = C_3 \oplus C_4$ (carry into MSB XOR carry out of MSB).

D2. Half subtractor and full subtractor

Half subtractor (A – B)

A	B	Difference	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Difference = $A \oplus B$ (same as adder Sum!)
 Borrow = $A' \cdot B$ (differs from adder Carry only by the inverter on A)

Memory hook: half subtractor = half adder with an inverter on A feeding the AND gate.

Full subtractor (A – B – Bin)

A	B	Bin	Diff	Bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Diff = $A \oplus B \oplus \text{Bin}$ (identical to full-adder Sum)
 Bout = $A'B + A' \cdot \text{Bin} + B \cdot \text{Bin}$
 = $A'B + (A \oplus B)' \cdot \text{Bin}$ (equivalent form)

Diff = $\Sigma m(1,2,4,7)$, Bout = $\Sigma m(1,2,3,7)$.

Comparison table (a favourite 5-mark question):

	Full Adder	Full Subtractor
Sum/Difference	$A \oplus B \oplus C$	$A \oplus B \oplus \text{Bin}$ (identical)
Carry/Borrow	$AB + BC + AC$	$A'B + A' \cdot \text{Bin} + B \cdot \text{Bin}$
Difference	—	A is complemented in the borrow expression

In practice **nobody builds subtractors** — you use an adder with 2's complement (§D1). Say this if asked "why".

D3. Decoders

Concept

A decoder turns a **coded input into a "one-hot" output**: exactly one of its output lines goes active for each input combination. Think of it as "which one of these 8 things did you name?" — this is precisely how a memory chip selects one word out of many from an address, and how an instruction opcode selects one control line in a CPU.

An **n-to-2ⁿ decoder** has n inputs and 2ⁿ outputs.

2-to-4 decoder with Enable

E	A1	A0	Y3	Y2	Y1	Y0
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

$$\begin{aligned} Y0 &= E \cdot A1' \cdot A0' & Y1 &= E \cdot A1' \cdot A0 \\ Y2 &= E \cdot A1 \cdot A0' & Y3 &= E \cdot A1 \cdot A0 \end{aligned}$$

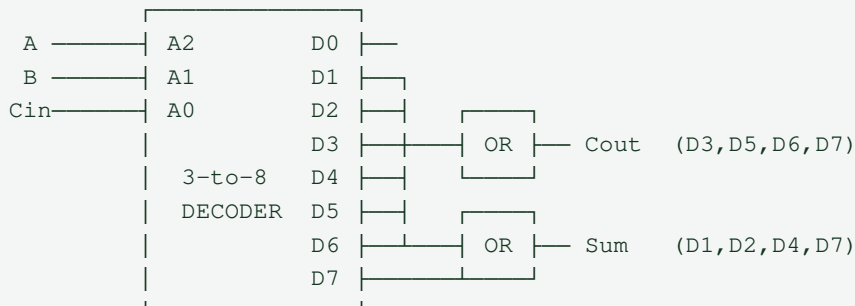
Notice: each output IS a minterm. That is the key to the next idea.

Implementing functions with a decoder

Because an n-to-2ⁿ decoder generates all 2ⁿ minterms, any function of n variables can be built as an OR of the appropriate decoder outputs.

Worked example 26. Implement a full adder using a 3-to-8 decoder.

$$\begin{aligned} \text{Sum} &= \Sigma m(1, 2, 4, 7) \rightarrow \text{OR together decoder outputs } D1, D2, D4, D7 \\ \text{Cout} &= \Sigma m(3, 5, 6, 7) \rightarrow \text{OR together decoder outputs } D3, D5, D6, D7 \end{aligned}$$



If the decoder has **active-LOW** outputs (as real 74138 chips do), replace the OR gates with **NAND** gates. Very common exam refinement.

Decoder expansion

Two 3-to-8 decoders + one inverter make a **4-to-16 decoder**: use the MSB as the enable — directly to one chip, inverted to the other.

Encoders — the inverse

An encoder has **2^n inputs and n outputs**; it outputs the binary code of whichever input line is active.

8-to-3 encoder (octal-to-binary):

$$\begin{aligned}Y_2 &= D_4 + D_5 + D_6 + D_7 \\Y_1 &= D_2 + D_3 + D_6 + D_7 \\Y_0 &= D_1 + D_3 + D_5 + D_7\end{aligned}$$

(Just OR together all input lines whose index has a 1 in that bit position.)

Two problems with a plain encoder — this is what gets examined:

1. **Ambiguity if two inputs are active at once** (D3 and D6 together would produce $111 = 7$, which is wrong for both).
2. **Output 000 is ambiguous**: it means either "D0 is active" or "no input is active."

Fixed with an extra **valid/V** output.

Priority encoder solves problem 1: if several inputs are active it encodes the **highest-priority (usually highest-numbered)** one and ignores the rest.

4-to-2 priority encoder truth table (D3 highest priority):

D3	D2	D1	D0	Y1	Y0	V
0	0	0	0	X	X	0
0	0	0	1	0	0	1
0	0	1	X	0	1	1
0	1	X	X	1	0	1
1	X	X	X	1	1	1

$$\begin{aligned}Y_1 &= D_3 + D_2 \\Y_0 &= D_3 + D_2'D_1 \\V &= D_3 + D_2 + D_1 + D_0\end{aligned}$$

Real-world use: interrupt priority resolution in a CPU (IC 74147 is a 10-to-4 priority encoder).

	Decoder	Encoder
Inputs → Outputs	$n \rightarrow 2^n$	$2^n \rightarrow n$
Direction	Code → one-hot	One-hot → code
Active outputs	Exactly one	The binary code
Typical use	Memory address selection, instruction decode, display drive	Keyboard scanning, interrupt priority

D4. Multiplexers and demultiplexers

Concept

A multiplexer is an **electronic rotary switch**. It has many data inputs, a few select lines, and **one output**; the select value decides which input gets connected through. It is a **data selector** — many-to-one. The classic reason to use one is to send many signals down one expensive wire (time-division multiplexing), but in digital design its *other* use is far more examinable: **a MUX can implement any Boolean function**.

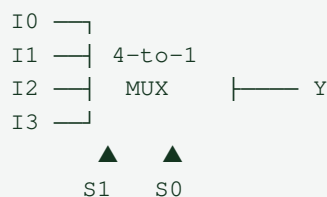
A **demultiplexer** is the same switch run backwards: one input, many outputs, select decides which output receives the data. One-to-many, a **data distributor**.

4-to-1 MUX

S1	S0	Y
0	0	I0
0	1	I1
1	0	I2
1	1	I3

$$Y = S1'S0' \cdot I0 + S1'S0 \cdot I1 + S1S0' \cdot I2 + S1S0 \cdot I3$$

A **2ⁿ-to-1 MUX** needs **n select lines**. (8-to-1 → 3 selects; 16-to-1 → 4 selects.)



Worked example 27 — implement a 3-variable function with a 4-to-1 MUX

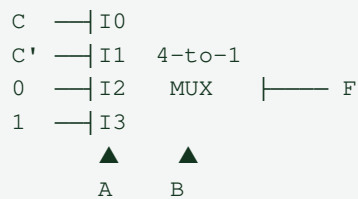
$$F(A,B,C) = \Sigma m(1, 2, 6, 7)$$

Method: use the two *most significant* variables (A, B) as the **select lines**. Then tabulate the function for each AB pair and express what's left in terms of C.

A	B	C	F	Select AB	Minterms	F values	Input needed
0	0	0	0	00	m0, m1	0, 1	F follows C → I0 = C
0	0	1	1				
0	1	0	1	01	m2, m3	1, 0	F is opposite of C → I1 = C'
0	1	1	0				

1	0	0	0	10	m4, m5	0, 0	always 0 → I2 = 0
1	0	1	0				
1	1	0	1	11	m6, m7	1, 1	always 1 → I3 = 1
1	1	1	1				

Answer: connect A→S1, B→S0, and I0=C, I1=C', I2=0, I3=1.



The general rule (state it in the exam): a **2ⁿ-to-1 MUX** can implement any function of **n+1 variables** — n variables go to the select lines and the remaining variable (or its complement, or 0, or 1) goes to the data inputs. So an 8-to-1 MUX handles 4 variables, a 16-to-1 handles 5.

Demultiplexer

A 1-to-4 DEMUX:

$$Y_0 = D \cdot S_1' S_0' \quad Y_1 = D \cdot S_1' S_0 \quad Y_2 = D \cdot S_1 S_0' \quad Y_3 = D \cdot S_1 S_0$$

Key insight for MCQs: a **decoder with an enable input IS a demultiplexer** — just treat the enable pin as the data input. That is why the 74138 is documented as a "decoder/demultiplexer."

Comparison table

	MUX	DEMUX	Decoder	Encoder
Inputs	2 ⁿ data + n select	1 data + n select	n	2 ⁿ
Outputs	1	2 ⁿ	2 ⁿ	n
Also called	Data selector	Data distributor	Minterm generator	—
Direction	Many→one	One→many	Code→one-hot	One-hot→code

D5. Comparators, parity, code converters

Magnitude comparator

1-bit comparator:

$$A > B : A \cdot B'$$

$$A < B : A' \cdot B$$

$$A = B : A \odot B = A'B' + AB \quad (\text{XNOR} - \text{equality is XNOR})$$

n-bit comparison works from the MSB down: the numbers are equal only if **all** bit-pairs are equal (AND of all XNORs); $A > B$ if the highest-order differing bit has $A=1$.

For 4-bit:

$$\begin{aligned} (A = B) &= x_3 \cdot x_2 \cdot x_1 \cdot x_0 & \text{where } x_i &= A_i \odot B_i \\ (A > B) &= A_3B_3' + x_3 \cdot A_2B_2' + x_3x_2 \cdot A_1B_1' + x_3x_2x_1 \cdot A_0B_0' \\ (A < B) &= A_3'B_3 + x_3 \cdot A_2'B_2 + x_3x_2 \cdot A_1'B_1 + x_3x_2x_1 \cdot A_0'B_0 \end{aligned}$$

IC **7485** is a 4-bit magnitude comparator (cascadable).

Parity generator / checker

- **Even parity generator** for 3 data bits: $P = A \oplus B \oplus C$ (makes total number of 1s even).
- **Odd parity generator:** $P = (A \oplus B \oplus C)'$.
- **Checker:** XOR all bits including parity. Result 1 $\Rightarrow$ error (for even parity).
- Detects **any odd number** of bit errors; **cannot detect an even number** and **cannot correct** anything. IC 74180 is a 9-bit parity generator/checker.

Code converters

The general procedure: write a truth table with the **source code as inputs** and the **target code as outputs**, then K-map each output bit.

Binary $\rightarrow$ Gray (4-bit):

$$\begin{aligned} G_3 &= B_3 \\ G_2 &= B_3 \oplus B_2 \\ G_1 &= B_2 \oplus B_1 \\ G_0 &= B_1 \oplus B_0 \end{aligned}$$

Three XOR gates. Very cheap.

Gray $\rightarrow$ Binary (4-bit):

$$\begin{aligned} B_3 &= G_3 \\ B_2 &= B_3 \oplus G_2 = G_3 \oplus G_2 \\ B_1 &= B_2 \oplus G_1 = G_3 \oplus G_2 \oplus G_1 \\ B_0 &= B_1 \oplus G_0 = G_3 \oplus G_2 \oplus G_1 \oplus G_0 \end{aligned}$$

Note this is a **cascade** (each output feeds the next) so it is slower.

BCD $\rightarrow$ Excess-3 (worked truth table, with 1010–1111 as don't-cares):

Decimal	B3 B2 B1 B0	E3 E2 E1 E0
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110

4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100
10–15	—	X (don't care)

Minimising each output with a K-map (using the six don't-cares) gives the standard result:

$$\begin{aligned}
 E3 &= B3 + B2B1 + B2B0 &= B3 + B2(B1 + B0) \\
 E2 &= B2'B1 + B2'B0 + B2B1'B0' &= B2'(B1 + B0) + B2B1'B0' = B2 \oplus (B1 + B0) \\
 E1 &= B1'B0' + B1B0 &= B1 \odot B0 \quad (\text{XNOR}) \\
 E0 &= B0'
 \end{aligned}$$

Spot check, input 5 = 0101: $E3 = 0 + 1 \cdot (0+1) = 1$; $E2 = 0 \oplus (0+1) = \dots$ **careful:** $B2=1$ so $E2 = B2 \oplus (B1+B0) = 1 \oplus 1 = 0$; $E1 = B1 \odot B0 = 0 \odot 1 = 0$; $E0 = B0' = 0 \rightarrow$ **1000** ✓ (Excess-3 for 5 is 1000).

BCD $\rightarrow$ 7-segment decoder: 4 inputs (BCD), 7 outputs (a–g). Segments are lit to form the digit. IC **7447** (common-anode, active-low outputs) / **7448** (common-cathode). Worth knowing the segment layout:



Digit 0 lights a,b,c,d,e,f (not g). Digit 1 lights only b,c. Digit 8 lights all seven.

Likely exam questions — Combinational circuits

Q	Marks
Distinguish between combinational and sequential circuits with examples.	5
Design a half adder and a half subtractor; give truth tables, expressions and logic diagrams.	5
Explain the difference between a decoder and a demultiplexer.	5
Design a full adder. Show how it can be built from two half adders and one OR gate.	10
Implement $F(A,B,C) = \Sigma m(1,2,6,7)$ using (i) a 3-to-8 decoder with OR gates and (ii) a 4-to-1 multiplexer.	10

Design a 4-bit binary adder-subtractor using full adders and XOR gates. Explain how the mode control line M selects between addition and subtraction, and how overflow is detected.	15
Explain the design procedure for combinational circuits. Using it, design a BCD-to-Excess-3 code converter, showing the truth table, K-maps for all four outputs (using don't-cares) and the final logic diagram.	15
Explain multiplexers. Show how a 2^n -to-1 MUX can realise any function of $n+1$ variables, and implement $F(A,B,C,D) = \Sigma m(0,1,3,4,8,9,15)$ using an 8-to-1 MUX.	15

MCQ traps — Combinational circuits

Trap	Truth
A 4-to-1 MUX has 4 select lines	It has 2 select lines ($2^n \rightarrow n$).
A decoder has n inputs and n outputs	n inputs, 2^n outputs.
An 8-to-1 MUX can implement 3-variable functions	It can implement 4-variable functions ($n+1$).
Half adder can be cascaded to add multi-bit numbers	It cannot — no carry input. That's the definition of "half".
Full subtractor difference $\neq$ full adder sum	They are identical : $A \oplus B \oplus C$. Only borrow/carry differ.
A ripple carry adder is fast	It is the slowest ; delay grows linearly with n. CLA fixes this.
Priority encoder and encoder are the same	Priority encoder resolves multiple simultaneous active inputs.
Decoder $\neq$ demultiplexer	A decoder with enable = demultiplexer .
Parity can correct a single-bit error	It can only detect an odd number of errors. Hamming corrects.
For n-bit binary $\rightarrow$ Gray you need n XOR gates	You need $n-1$ XORs (the MSB passes through unchanged).

PART E — SEQUENTIAL CIRCUITS

E1. Latches and flip-flops

Concept

A combinational circuit forgets everything the instant its inputs change. To build a counter, a register, or a memory you need a circuit that **remembers**. The trick is **feedback**: connect the output of a gate back to its own input and the circuit becomes bistable — it can rest in either of two states indefinitely, and that's one bit stored.

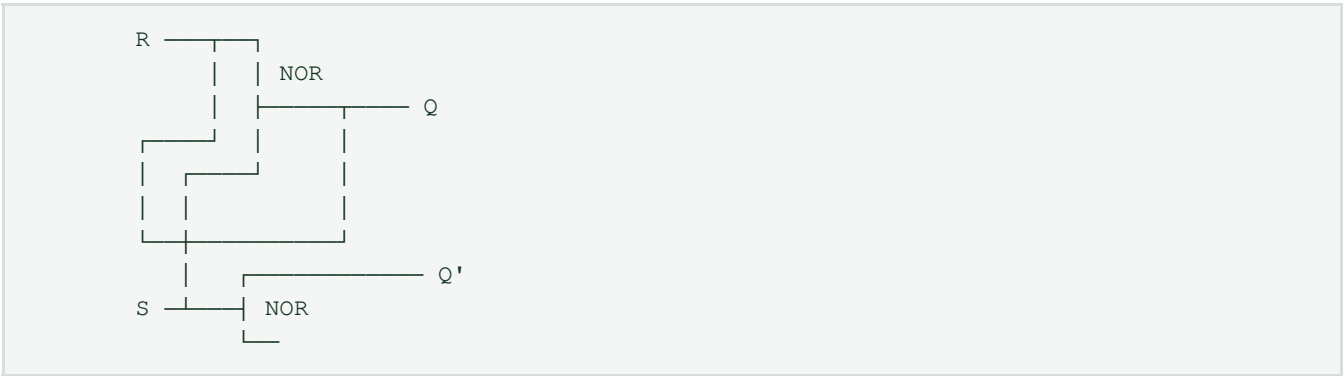
The simplest such device is the **SR latch** made from two cross-coupled NOR (or NAND) gates. A latch is **level-sensitive** — while the enable is high, the output keeps tracking the input. That's often undesirable, because in a big synchronous system you want *every* storage element to change at exactly the same instant, so timing is predictable. Adding **edge-triggering** — the device samples its input only at the rising (or falling) edge of

a clock — turns a latch into a **flip-flop**.

Latch vs flip-flop — a 5-mark table

	Latch	Flip-flop
Triggering	Level-sensitive (transparent while enable is active)	Edge-triggered (samples at the clock edge only)
Clock	May be unlocked or enable-driven	Clocked
Transparency	Transparent — output follows input	Opaque — output changes only at the edge
Speed / area	Faster, fewer gates	Slower, more gates
Use	Simple storage, transparent buffers	Synchronous systems, registers, counters

SR latch (NOR-based)



S	R	Q(next)	Comment
0	0	Q (no change)	Hold / memory state
0	1	0	Reset
1	0	1	Set
1	1	?	FORBIDDEN / invalid — both Q and Q' go 0, and on releasing to 00 the next state is unpredictable (race)

For a **NAND-based SR latch (S'R' latch)**, the inputs are **active low** and the forbidden condition is **S=R=0**. Examiners swap these deliberately.

The four flip-flops — characteristic tables and equations

Characteristic table answers "given the inputs and the present state, what is the next state?" (analysis direction). **Excitation table** answers "I want this state transition — what inputs must I apply?" (design direction). You need both.

SR flip-flop

S	R	Q(t+1)	
---	---	--------	--

0	0	Q(t)	No change
0	1	0	Reset
1	0	1	Set
1	1	invalid	Forbidden

Characteristic equation: $Q(t+1) = S + R' \cdot Q(t)$, with constraint $S \cdot R = 0$.

JK flip-flop

Concept: the JK is the SR with the illegal combination redefined to something useful — **toggle**. It is therefore the most versatile flip-flop (J and K are said to be named after Jack Kilby).

J	K	Q(t+1)	
0	0	Q(t)	No change
0	1	0	Reset
1	0	1	Set
1	1	Q(t)'	Toggle

Characteristic equation: $Q(t+1) = J \cdot Q'(t) + K' \cdot Q(t)$

D flip-flop (Data / Delay)

D	Q(t+1)
0	0
1	1

Characteristic equation: $Q(t+1) = D$

Simplest to use; the standard building block of registers, since the data just appears at the output one clock later (hence "delay"). Built from an SR/JK by tying $R = S'$ ($K = J'$).

T flip-flop (Toggle)

T	Q(t+1)	
0	Q(t)	Hold
1	Q(t)'	Toggle

Characteristic equation: $Q(t+1) = T \oplus Q(t) = T \cdot Q'(t) + T' \cdot Q(t)$

Built from a JK by tying $J = K = T$. Ideal for counters.

The EXCITATION TABLES — memorise this block

This single table is the most-used object in the whole sequential-design syllabus.

$Q(t) \rightarrow Q(t+1)$	S R	J K	D	T
$0 \rightarrow 0$	0 X	0 X	0	0
$0 \rightarrow 1$	1 0	1 X	1	1
$1 \rightarrow 0$	0 1	X 1	0	1
$1 \rightarrow 1$	X 0	X 0	1	0

Memory hooks:

- **JK is all X's on the diagonal pattern:** J is specified (0/1) when starting from 0 and don't-care when starting from 1; K is the mirror image. JK gives the **most don't-cares**, which is exactly why JK-based designs usually produce the simplest logic.
- **D = the next state**, always. Trivially easy but produces no don't-cares.
- **T = 1 whenever the state changes** ($T = Q \oplus Q^+$).

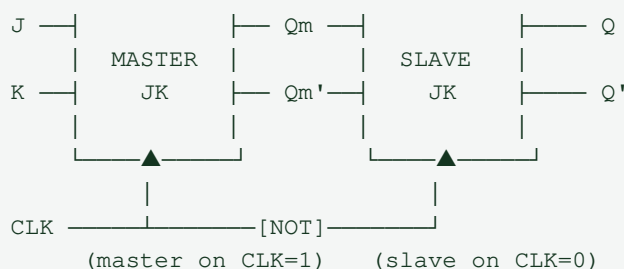
Race-around condition and master-slave

The problem: in a **level-triggered** JK flip-flop with $J = K = 1$, the output toggles. But the output feeds back to the input, so as long as the clock is high it toggles *again*, and again — oscillating. If the clock pulse width t_p is greater than the propagation delay t_{pd} , the output races around unpredictably. The final state when the clock falls is indeterminate. **This is the race-around condition.**

Three solutions:

1. Make the clock pulse narrower than the propagation delay ($t_p < t_{pd}$) — impractical.
2. **Master-slave configuration** — two flip-flops in series. The master is enabled while $CLK = 1$ and captures the input; the slave is enabled while $CLK = 0$ and copies the master out. Since the two are **never enabled simultaneously**, feedback can never race. The output changes on the *falling* edge (**pulse-triggered**).
3. **Edge-triggering** — the modern solution: the device samples only during the tiny instant of the clock transition.

MASTER-SLAVE JK — hand-draw this



Timing parameters (MCQ material)

Term	Meaning
Setup time (t_{su})	Minimum time the data input must be stable BEFORE the clock edge

Hold time (t_h)	Minimum time the data must remain stable AFTER the clock edge
Propagation delay (t_{pd})	Clock edge to valid output
Metastability	If setup/hold is violated the output can hover at an invalid level for an unbounded time
Maximum clock frequency	$f_{\text{max}} = 1 / (t_{\text{pd}} + t_{\text{combinational}} + t_{\text{su}})$

Asynchronous inputs: PRESET (forces Q=1) and CLEAR/RESET (forces Q=0) override the clock entirely and act immediately. They are used to initialise a circuit at power-up.

E2. Registers and shift registers

Concept

A **register** is just n flip-flops sharing a common clock, storing an n -bit word. Add the ability to move the bits sideways on each clock and you have a **shift register** — which is how you do serial-to-parallel conversion (UART receive), parallel-to-serial conversion (UART transmit), multiplication and division by 2, and delay lines.

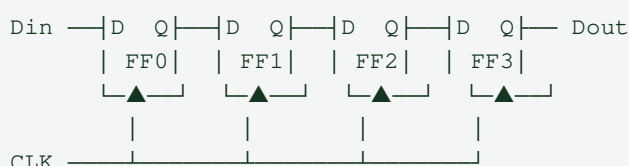
Shifting left by one position multiplies by 2; shifting right by one divides by 2.

(For signed numbers use an **arithmetic** right shift, which replicates the sign bit, rather than a **logical** right shift, which brings in 0.)

The four shift-register types

Type	Data in	Data out	Clocks to load n bits	Clocks to read n bits	Use
SISO (Serial In Serial Out)	Serial	Serial	n	n	Delay line
SIPO (Serial In Parallel Out)	Serial	Parallel	n	1 (immediate)	Serial→parallel conversion (receiving)
PISO (Parallel In Serial Out)	Parallel	Serial	1	n	Parallel→serial conversion (transmitting)
PIPO (Parallel In Parallel Out)	Parallel	Parallel	1	1	Simple storage register (no shifting needed)

4-BIT SISO SHIFT REGISTER (right shift)



Universal shift register

Can do **all** of: shift left, shift right, parallel load, and hold. Needs 2 mode-select lines (S1, S0) and a 4-to-1 MUX in front of each flip-flop:

S1	S0	Operation
0	0	Hold (no change)
0	1	Shift right
1	0	Shift left
1	1	Parallel load

IC **74194** is the standard 4-bit universal shift register.

Ring counter and Johnson counter

These are shift registers with the output fed back to the input — a nice bridge topic between registers and counters.

	Ring counter	Johnson (twisted-ring / switch-tail) counter
Feedback	Q of last FF → D of first FF	Q' of last FF → D of first FF (inverted)
n flip-flops give	n states	2n states
Initial state	Must be preset to 1000 (one 1 circulating)	Starts at 0000
Decoding	No decoding gates needed — each FF output is a state	Needs a 2-input AND per state
Efficiency	Poor (n states from 2 ⁿ possible)	Better (2n states)

4-bit ring counter sequence: 1000 → 0100 → 0010 → 0001 → 1000 ... (**4 states**)

4-bit Johnson counter sequence (8 states):

0000 → 1000 → 1100 → 1110 → 1111 → 0111 → 0011 → 0001 → back to 0000

Neither is self-starting — an illegal state can circulate forever unless correction logic is added. Say this if asked for a disadvantage.

E3. Counters

Concept

A counter is a register that goes through a **predetermined sequence of states** on successive clock pulses. The two families differ in one thing only: **where the clock comes**

from.

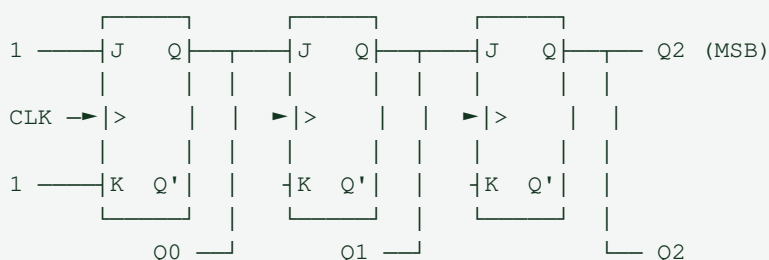
- In an **asynchronous (ripple) counter**, only the first flip-flop gets the real clock. Each subsequent flip-flop is clocked by the *output* of the one before it. Simple, few gates — but each stage's change has to *ripple* through, so delays accumulate and there are brief intervals when the count output is garbage ("decoding glitches").
- In a **synchronous counter**, **every flip-flop is clocked simultaneously** by the same clock, and combinational logic on the J/K/T inputs decides which ones toggle. More gates, but all outputs change together, so it's fast and glitch-free.

Comparison — a guaranteed exam table

	Asynchronous (Ripple)	Synchronous
Clock	Only FF0 gets the system clock; others clocked by preceding FF output	All FFs share the same clock
Speed	Slow — delay = $n \times t_{pd}$	Fast — delay = t_{pd} (one stage)
Max frequency	$f = 1/(n \times t_{pd})$	$f = 1/(t_{pd} + t_{gate})$
Circuit complexity	Simple , minimal extra gates	More combinational logic needed
Glitches / spikes on decoded outputs	Yes (transient false states while rippling)	No
Design method	Ad hoc / by inspection	Formal state-table + excitation-table procedure
Cost	Low	Higher
Example IC	7493	74163

Asynchronous (ripple) up counter

3-BIT RIPPLE UP COUNTER (negative-edge triggered, J=K=1 on all)



Each stage divides the frequency by 2, so with n flip-flops the last output is $f_{\text{clock}} / 2^n$. Hence a counter is also a **frequency divider**.

- For a **DOWN counter**: either clock the next stage from **Q'** instead of Q (with negative-edge FFs), or use positive-edge FFs clocked from Q.

Mod-N counters

Number of flip-flops needed for a mod-N counter: the smallest n such that $2^n \geq N$.

Mod (N)	Flip-flops	States used	States skipped
8	3	0–7	0
10 (decade)	4	0–9	6
12	4	0–11	4
16	4	0–15	0
60	6	0–59	4
100	7	0–99	28

Worked example 28 — design a MOD-10 (decade) ripple counter.

Need 4 FFs ($2^4 = 16 \geq 10$). Count 0000 ... 1001, then reset to 0000 on 1010.

Method: detect the FIRST unwanted state (1010 = 10) and feed the detection into the asynchronous CLEAR of every flip-flop.

Detection: 1010 has $Q_3=1$ and $Q_1=1$ (Q_2 and Q_0 are 0, but $Q_3 \cdot Q_1$ is unique among 0000–1010).

$\text{CLEAR} = (Q_3 \cdot Q_1)'$ [active-LOW clear, so use a NAND gate]



CLR	CLR	CLR	CLR	← all four flip-flops cleared simultaneously
FF0	FF1	FF2	FF3	

Sequence: 0...9, momentarily 1010, instantly cleared to 0000. Ten distinct states → MOD-10 ✓

State the known drawback: the state 1010 does exist for a few nanoseconds — a **transient glitch**. Also, if the flip-flops clear at slightly different speeds the counter can misbehave. The synchronous approach (below) has neither problem. IC **7490** is a decade counter built this way.

Synchronous counter design — the FULL procedure

The six steps (write these out; they are worth marks in themselves):

1. Draw the **state diagram** for the required sequence.
2. Determine the **number of flip-flops**: n where $2^n \geq \text{number of states}$. Assign state codes.
3. Draw the **state (transition) table**: present state → next state.
4. Choose a flip-flop type and use its **excitation table** to fill in the required input

values for every transition (unused states → don't-cares).

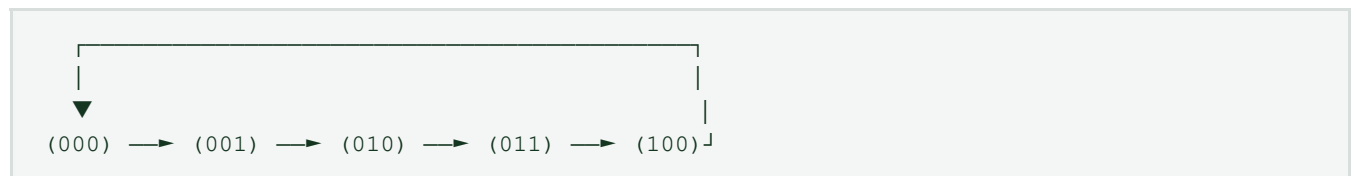
5. **K-map each flip-flop input** as a function of the present state variables and simplify.

6. Draw the **logic diagram**; verify, and check the circuit is **self-starting**.

Worked example 29 — synchronous MOD-5 counter using T flip-flops

Required sequence: 000 → 001 → 010 → 011 → 100 → 000 (repeat).

Step 1 — State diagram:



Step 2: 5 states → $2^3 = 8 \geq 5$ → **3 flip-flops** (Q2 Q1 Q0). States 101, 110, 111 unused → **don't-cares**.

Steps 3 & 4 — State table with T excitation (recall: $T = Q \oplus Q^+$, i.e. $T = 1$ iff the bit changes):

Present Q2 Q1 Q0	Next Q2 ⁺ Q1 ⁺ Q0 ⁺	T2	T1	T0
0 0 0	0 0 1	0	0	1
0 0 1	0 1 0	0	1	1
0 1 0	0 1 1	0	0	1
0 1 1	1 0 0	1	1	1
1 0 0	0 0 0	1	0	0
1 0 1	— unused —	X	X	X
1 1 0	— unused —	X	X	X
1 1 1	— unused —	X	X	X

Step 5 — K-maps (3-variable maps, Q2 on rows, Q1Q0 on columns 00/01/11/10):

T0:

	Q1Q0				
Q2 \	00	01	11	10	
0	1	1	1	1	← m0, m1, m3, m2 all 1
1	0	X	X	X	← m4=0, m5, m7, m6 = X

The whole Q2=0 row is 1 → **T0 = Q2'**.

(Could we group the Xs with the top row to get $T0 = 1$? No — $m4$ is a genuine 0, so no.)

T1:

	Q1Q0				
Q2 \	00	01	11	10	

0		0		1		1		0			← m1=1, m3=1
1		0		X		X		X			

Column pair Q0=1 (cells m1, m3, m5, m7) — m5 and m7 are don't-cares, so we may include them. That quad is defined by **Q0 = 1 → T1 = Q0**.

T2:

							Q1Q0		
Q2 \	00	01	11	10					
0	0	0	1	0				← m3 = 1	
1	1	X	X	X				← m4 = 1, rest don't-care	

- m4 with don't-cares m5, m7, m6 → the whole bottom row → **Q2**
- m3 alone → Q2'Q1Q0; but pair it with don't-care m7 → **Q1Q0**

T2 = Q2 + Q1·Q0

Final equations:

```
T2 = Q2 + Q1Q0
T1 = Q0
T0 = Q2'
```

Step 6 — Verification (do this in the exam; it earns credit):

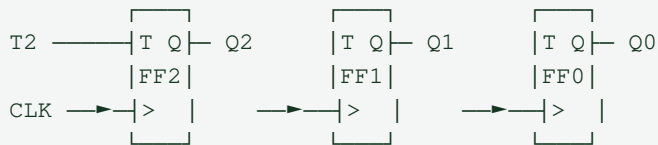
Present	T2 T1 T0	Toggles applied	Next state	Correct?
000	0, 0, 1	Q0 toggles	001	✓
001	0, 1, 1	Q1, Q0 toggle	010	✓
010	0, 0, 1	Q0 toggles	011	✓
011	1, 1, 1	all toggle	100	✓
100	1, 0, 0	Q2 toggles	000	✓

Self-starting check (an examiner's favourite follow-up):

Unused state	T2 T1 T0	Next state	Lands in valid sequence?
101	1, 1, 0	011	✓
110	1, 0, 1	011	✓
111	1, 1, 0	001	✓

The counter is fully self-starting — every illegal state returns to the main sequence within one clock. State this conclusion explicitly.

Logic diagram to hand-draw:



$$\begin{aligned}
 T2 &= Q2 + (Q1 \text{ AND } Q0) && [1 \text{ AND gate} + 1 \text{ OR gate}] \\
 T1 &= Q0 && [\text{direct wire}] \\
 T0 &= Q2' && [1 \text{ inverter}]
 \end{aligned}$$

Worked example 30 — synchronous 3-bit up counter using JK flip-flops

For a **full-modulus binary up counter** there is a beautiful shortcut you should memorise:

$$\begin{aligned}
 J0 &= K0 = 1 \\
 J1 &= K1 = Q0 \\
 J2 &= K2 = Q0 \cdot Q1 \\
 J3 &= K3 = Q0 \cdot Q1 \cdot Q2
 \end{aligned}$$

In plain English: a bit toggles when **ALL lower-order bits are 1**. (That's exactly how carrying works in binary counting.) This generalises to any width. For a **down counter**, use the complements: $J1 = K1 = Q0'$, $J2 = K2 = Q0'Q1'$, etc.

Verify at state 011: $J2=K2 = Q0 \cdot Q1 = 1 \rightarrow Q2$ toggles $0 \rightarrow 1$; $J1=K1 = Q0 = 1 \rightarrow Q1$ toggles $1 \rightarrow 0$; $J0=K0=1 \rightarrow Q0$ toggles $1 \rightarrow 0$. Next state = 100 ✓

E4. Mealy and Moore machines, state diagrams

Concept

A **finite state machine** is the formal model of any sequential circuit: a finite set of states, a transition function, and an output function. The one design decision that defines the two flavours is **what the output depends on**.

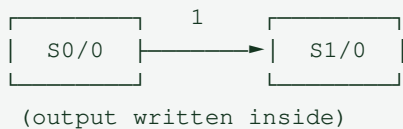
- **Moore machine:** output depends **only on the present state**. Draw the output *inside the state bubble*.
- **Mealy machine:** output depends on **the present state AND the current input**. Draw the output *on the transition arrow*, written **input / output**.

Comparison

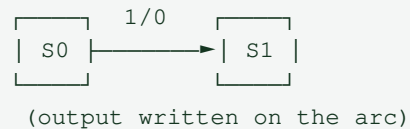
	Moore	Mealy
Output is a function of	State only	State and input
Output shown on the diagram	Inside the state circle	On the arrow, as input/output

Number of states needed	Generally more	Generally fewer
Output timing	Changes only on the clock edge — synchronous, glitch-free	Can change asynchronously the moment the input changes
Response to an input	Delayed by one clock cycle	Immediate (faster)
Susceptible to input glitches	No	Yes
Typical model	Output = f(state)	Output = f(state, input)

MOORE (1101 sequence detector, partial)

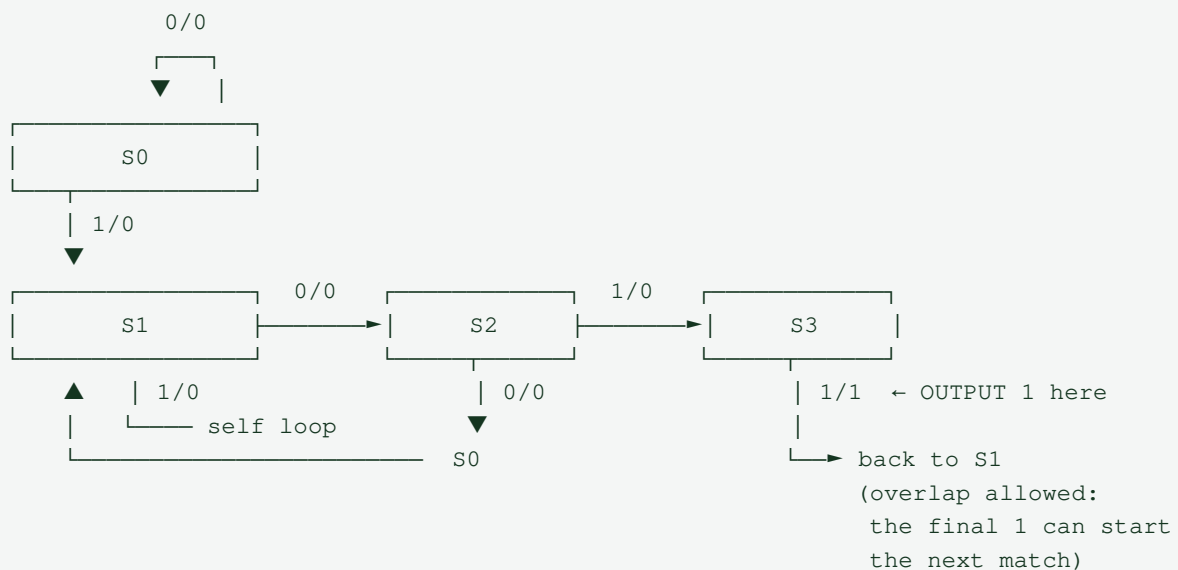


MEALY (same detector, partial)



Worked example 31 — Mealy state diagram for a 1011 sequence detector (overlapping)

States: S0 = nothing matched, S1 = "1", S2 = "10", S3 = "101"



State table:

Present	Input 0 → (Next, Out)	Input 1 → (Next, Out)
S0	S0, 0	S1, 0

S1	S2, 0	S1, 0
S2	S0, 0	S3, 0
S3	S2, 0	S1, 1

(The S3-on-0 transition goes to S2 because "1011" + "0" ends in "10", which is state S2 — this is the subtlety of overlapping detectors.)

State reduction

Two states are **equivalent** if, for every possible input sequence, they produce the same output sequence and go to equivalent next states. Equivalent states can be merged, reducing the flip-flop count. The systematic tool is the **implication chart** / partitioning method. Reducing states from, say, 7 to 4 drops you from 3 flip-flops to 2.

Analysis of a given sequential circuit (the reverse direction — also examinable)

1. Write the **flip-flop input equations** by reading the circuit.
2. Substitute them into the flip-flop's **characteristic equation** to get the **next-state equations**.
3. Build the **state table**.
4. Draw the **state diagram**.

Worked example 32. A D flip-flop circuit has $D_A = A \oplus x$, $D_B = A'x$, and output $y = AB$.

Next-state equations: $A^+ = A \oplus x$; $B^+ = A'x$

A B	x=0: A^+B^+ , y	x=1: A^+B^+ , y
0 0	00, 0	11, 0
0 1	00, 0	11, 0
1 0	10, 0	00, 0
1 1	10, 1	00, 1

(This is a **Moore** machine — y depends only on A and B, not on x.)

Likely exam questions — Sequential circuits

Q	Marks
Differentiate between a latch and a flip-flop.	5
Write the characteristic table, characteristic equation and excitation table of the JK flip-flop.	5
What is the race-around condition? How does the master-slave configuration eliminate it?	5
Differentiate between Mealy and Moore machines with a diagram of each.	5

Explain the four types of shift register (SISO, SIPO, PISO, PIPO) and give one application of each. Draw a 4-bit SIPO register.	10
Compare synchronous and asynchronous counters. Design a MOD-10 ripple counter and explain how the reset logic works.	10
Distinguish between a ring counter and a Johnson counter. Draw both for 4 bits and give their state sequences.	10
Explain the design procedure for synchronous sequential circuits. Using it, design a synchronous MOD-5 counter with T flip-flops: state diagram, state table, excitation table, K-maps, logic diagram, and verification that the design is self-starting.	15
Explain the working of SR, JK, D and T flip-flops with truth tables, characteristic equations, excitation tables and logic diagrams. Show how a JK flip-flop can be converted into (i) a D flip-flop and (ii) a T flip-flop.	15

MCQ traps — Sequential circuits

Trap	Truth
SR flip-flop forbidden state is $S=R=0$	For a NOR-based SR latch it's $S=R=1$. For a NAND-based latch it's $S=R=0$ (active low). Read which one.
JK with $J=K=1$ sets the output to 1	It toggles .
D flip-flop needs 2 inputs	One data input; it's the SR/JK with $R = S'$.
Excitation for JK, $1 \rightarrow 0$ is $J=0$, $K=1$	It's $J = X$, $K = 1$.
Excitation for JK, $0 \rightarrow 1$ is $J=1$, $K=0$	It's $J = 1$, $K = X$.
Mod-10 counter needs 10 flip-flops	4 ($2^4 = 16 \geq 10$).
An n-bit ring counter has $2n$ states	n states. It's the Johnson counter that has $2n$.
Ripple counters are faster	Synchronous counters are faster; ripple delay accumulates.
A counter with 4 FFs can count to 16	It has 16 states, 0 to 15 . Highest count is 15.
Setup time is after the clock edge	Setup is before ; hold is after.
Moore output depends on input	Moore = state only . Mealy = state + input.
Shift left divides by 2	Shift left multiplies by 2; shift right divides.

Master-slave FF is edge triggered

It is **pulse (level) triggered** — output changes on the trailing edge, but it is not a true edge-triggered device.

PART F — LAST-WEEK REVISION SHEET

The 25 facts most likely to appear as one-mark MCQs

1. Hex A=10, B=11, C=12, D=13, E=14, F=15.
2. 1 hex digit = 4 bits; 1 octal digit = 3 bits.
3. Largest unsigned n-bit value = $2^n - 1$.
4. BCD invalid codes: **1010–1111 (six of them)**. Correction factor = **6 (0110)**.
5. ASCII is **7 bits, 128 characters**; EBCDIC is 8 bits.
6. **A** = 65, **a** = 97, **0** = 48. Case difference = **32**.
7. 8-bit 2's complement range: **-128 to +127**.
8. 2's complement has **one** zero; 1's complement and sign-magnitude have **two**.
9. 1's complement: **add the end-around carry**. 2's complement: **discard the carry**.
10. Overflow = same-signed operands giving an opposite-signed result = $C_{in}(MSB) \oplus C_{out}(MSB)$.
11. IEEE 754 single: **1 + 8 + 23**, bias **127**. Double: **1 + 11 + 52**, bias **1023**.
12. Exponent all 1s + fraction 0 = **infinity**; all 1s + non-zero fraction = **NaN**.
13. **$A + A'B = A + B$; $A + BC = (A+B)(A+C)$** .
14. De Morgan: $(AB)' = A' + B'$; $(A+B)' = A'B'$.
15. **NAND and NOR are universal**. XOR = 4 NANDs = 5 NORs.
16. K-map axes use **Gray code**; groups are powers of 2; corners wrap.
17. A group of 2^k cells eliminates **k** variables.
18. **Essential** prime implicants must be in the answer; ordinary prime implicants need not.
19. Decoder: $n \rightarrow 2^n$. Encoder: $2^n \rightarrow n$. MUX: **2^n data + n select $\rightarrow 1$** .
20. A **2^n -to-1 MUX** implements any function of **n+1** variables.
21. **Decoder + enable = demultiplexer**.
22. Full adder Sum = $A \oplus B \oplus C$ = full **subtractor** Difference. Only the carry/borrow differ.
23. JK excitation: $0 \rightarrow 0$ is (0,X); $0 \rightarrow 1$ is (1,X); $1 \rightarrow 0$ is (X,1); $1 \rightarrow 1$ is (X,0).
24. **$T = Q \oplus Q^+$; $D = Q^+$** ; JK char. eq. **$Q^+ = JQ' + K'Q$** ; SR char. eq. **$Q^+ = S + R'Q$** .
25. Ring counter with n FFs = **n** states; Johnson = **2n** states; binary counter = **2^n** .

Formulae to have on the tip of your pen

Value in base b	$= \sum d_i \cdot b^i$
2's complement	= 1's complement + 1 (or: copy up to first 1 from right, invert rest)
Signed value (2's)	$= -d(n-1) \cdot 2^{(n-1)} + \sum d_i \cdot 2^i$

IEEE 754 value	$= (-1)^S \times 1.F \times 2^{(E - \text{bias})}$	
Half adder	$S = A \oplus B$	$C = AB$
Full adder	$S = A \oplus B \oplus \text{Cin}$	$\text{Cout} = AB + \text{Cin}(A \oplus B)$
Half subtractor	$D = A \oplus B$	$\text{Bout} = A'B$
Full subtractor	$D = A \oplus B \oplus \text{Bin}$	$\text{Bout} = A'B + A'\text{Bin} + B \cdot \text{Bin}$
Char. equations	SR: $Q^+ = S + R'Q$ JK: $Q^+ = JQ' + K'Q$ D: $Q^+ = D$ T: $Q^+ = T \oplus Q$	
Mod-N counter	smallest n with $2^n \geq N$	
Sync. up counter	$J_i = K_i = Q_0 \cdot Q_1 \cdot \dots \cdot Q_{i-1}$	
Ripple delay	$n \times t_{pd}$	$f_{\max} = 1/(n \cdot t_{pd})$
Overflow	$V = C_{in}(\text{MSB}) \oplus C_{out}(\text{MSB})$	

Exam-day tactics for this topic

- **In the MCQ paper**, number-system conversions and complement arithmetic are the highest confidence-per-second questions in the whole paper. Do them first, bank the marks, and spend the saved time on the topics where you might have to leave a blank (remember: with $-1/3$ marking, a blank costs 0 and a wrong guess costs 0.667 marks — **only guess if you can eliminate at least two options**).
- **In the descriptive papers**, always: (a) draw the truth table, (b) draw the K-map, (c) write the simplified expression, (d) **draw the logic diagram**. Examiners allocate marks per component. A correct answer with no diagram loses easy marks; a diagram with a small algebra slip still scores.
- **Always verify** a counter or sequential design by walking through the states, and always comment on **self-starting**. It takes 90 seconds and reads as mastery.
- Label every K-map axis and every group. Unlabelled groups cannot be marked.